# LAPORAN MINI PROJECT

**MATA KULIAH KECERDASAN BUATAN**

(ARTIFICIAL INTELLIGENCE)

Ujian Akhir Semester (UAS)

# PERENCANAAN RUTE ROBOT PEMBERSIH DI RUANG KELAS MENGGUNAKAN ALGORITMA A* DENGAN SIMULASI 3D

*(Python, Blender, dan three.js)*


Disusun oleh:

**Kelompok 6 — Kelas AI-D**

Rafiq Alhariri Andriansyah (240605110178)

Ahmad Zamroni (240605110216)

Afzaal Abrar Hidayat (240605110206)

Dosen Pengampu:

[Nama Dosen Pengampu]

**UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM MALANG**

[Program Studi / Fakultas]


**2026**

# KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga Laporan Mini Project mata kuliah Kecerdasan Buatan (*Artificial Intelligence*) yang berjudul "Perencanaan Rute Robot Pembersih di Ruang Kelas Menggunakan Algoritma A* dengan Simulasi 3D" ini dapat diselesaikan.

Laporan ini merupakan kelanjutan dari proposal pada Ujian Tengah Semester (UTS) yang memuat latar belakang, tujuan, dan metode berupa PEAS serta formulasi masalah. Pada Ujian Akhir Semester (UAS), rancangan tersebut diwujudkan menjadi sistem yang utuh: inti kecerdasan buatan dalam bahasa Python, aset tiga dimensi yang dibuat secara terprogram dengan Blender, serta simulasi tiga dimensi *real-time* berbasis three.js yang memperlihatkan proses pencarian dan eksekusi rencana robot.

Penulis mengucapkan terima kasih kepada dosen pengampu mata kuliah Kecerdasan Buatan atas bimbingan dan ilmu yang diberikan, serta kepada seluruh anggota Kelompok 6 yang telah bekerja sama menyelesaikan proyek ini. Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Kritik dan saran yang membangun sangat diharapkan demi penyempurnaan di masa mendatang. Semoga laporan ini bermanfaat bagi pembaca.

Malang, November 2026
Penulis,

**Kelompok 6**

# DAFTAR ISI

KATA PENGANTAR ..... i
DAFTAR ISI ..... ii
DAFTAR GAMBAR ..... iv
DAFTAR TABEL ..... v
ABSTRAK ..... vi
ABSTRACT ..... vii
BAB I PENDAHULUAN ..... 1
1.1 Latar Belakang ..... 1
1.2 Rumusan Masalah ..... 2
1.3 Tujuan ..... 2
1.4 Manfaat ..... 3
1.5 Batasan Masalah ..... 3
1.6 Sistematika Penulisan ..... 3
BAB II LANDASAN TEORI ..... 5
2.1 Agen Cerdas dan Lingkungan ..... 5
2.1.1 Konsep Agen ..... 5
2.1.2 PEAS ..... 5
2.1.3 Sifat Lingkungan Tugas ..... 6
2.1.4 Jenis Agen ..... 6
2.2 Agen Pemecah Masalah ..... 6
2.2.1 Siklus Agen ..... 6
2.2.2 Formulasi Masalah ..... 7
2.3 Algoritma Pencarian ..... 8
2.3.1 Kriteria Penilaian ..... 8
2.3.2 Breadth-First Search (BFS) ..... 8
2.3.3 Uniform-Cost Search (UCS) ..... 8
2.3.4 Algoritma A* ..... 8
2.3.5 Greedy Best-First Search dan Nearest-Neighbor ..... 9
2.4 Fungsi Heuristik ..... 10
2.4.1 Admissible dan Consistent ..... 10
2.4.2 Heuristik untuk Pembersihan Multi-Target ..... 10
2.5 Teknologi Pendukung ..... 11
2.5.1 Python, FastAPI, dan Pustaka Analisis ..... 11
2.5.2 Blender dan bpy ..... 12
2.5.3 three.js ..... 12
2.6 Penelitian Terkait ..... 12
BAB III METODOLOGI DAN PERANCANGAN ..... 13
3.1 Tahapan Pengembangan ..... 13
3.2 Analisis Tugas dengan PEAS ..... 14
3.3 Formulasi Masalah ..... 14
3.3.1 Representasi State ..... 14
3.3.2 Lima Komponen Formulasi ..... 15

3.4 Skenario Pengujian ..... 15
3.5 Algoritma dan Heuristik yang Diuji ..... 17
3.6 Metrik Evaluasi ..... 17
3.7 Arsitektur Sistem ..... 18
3.8 Perancangan Simulasi 3D ..... 19
3.8.1 Pemetaan Koordinat dan Aset ..... 19
3.8.2 Alur Sutradara dan Antarmuka ..... 19
3.8.3 Visualisasi Pencarian, Kamera, dan Interaksi ..... 19
BAB IV IMPLEMENTASI, HASIL, DAN PEMBAHASAN ..... 21
4.1 Lingkungan Implementasi ..... 21
4.2 Implementasi Inti AI (Python) ..... 21
4.3 Implementasi Aset 3D dengan Blender ..... 23
4.4 Implementasi Simulasi three.js ..... 26
4.4.1 Rendering Realistis ..... 26
4.4.2 Ruang, Robot, dan Efek ..... 27
4.4.3 Visualisasi Proses Pencarian ..... 27
4.4.4 Mode Interaktif ..... 29
4.5 Pengujian Fungsional ..... 30
4.6 Hasil Eksperimen ..... 31
4.6.1 Hasil pada Skenario S1–S4 ..... 31
4.6.2 Pengaruh Heuristik ..... 33
4.6.3 Kualitas Rencana Heuristik Rakus ..... 34
4.6.4 Peta Panas Ekspansi State ..... 35
4.6.5 Uji Skalabilitas ..... 35
4.6.6 Penyusunan Ulang Rencana (Replanning) ..... 37
4.7 Pembahasan ..... 37
4.8 Keterbatasan ..... 39
BAB V PENUTUP ..... 40
5.1 Kesimpulan ..... 40
5.2 Saran ..... 41
DAFTAR PUSTAKA ..... 42
LAMPIRAN A PETUNJUK MENJALANKAN APLIKASI ..... 43
A.1 Instalasi dan Menjalankan ..... 43
A.2 Kontrol Aplikasi ..... 43
A.3 Perintah Pengujian dan Pembangunan Ulang ..... 43
LAMPIRAN B STRUKTUR PROYEK ..... 44
LAMPIRAN C HASIL BENCHMARK LENGKAP ..... 45
LAMPIRAN D CUPLIKAN KODE INTI ..... 46

# DAFTAR GAMBAR

Gambar 2.1 Interaksi agen dengan lingkungan dan ringkasan PEAS. ..... 5
Gambar 2.2 Siklus agen pemecah masalah beserta penyusunan ulang rencana. .....7
Gambar 3.1 Tahapan pengembangan proyek. ..... 13
Gambar 3.2 Denah skenario S1–S3 (rancangan UTS) beserta rute optimal A*. ...16
Gambar 3.3 Denah skenario S4 (kelas penuh 10 × 7) beserta rute optimal A*. ....16
Gambar 3.4 Arsitektur sistem CleanBot 3D. ..... 18
Gambar 4.1 Model robot CleanBot hasil skrip Blender (depan, belakang, samping, atas). .....24
Gambar 4.2 Properti ruang kelas dan model sampah hasil skrip Blender. ..... 24
Gambar 4.3 Render Cycles skenario S3 dengan rute rencana A* dan titik kotoran.25
Gambar 4.4 Render Cycles skenario S4 dengan robot di tengah rute. ..... 25
Gambar 4.5 Detail robot CleanBot pada render Cycles (kiri: S3, kanan: S4). ..... 26
Gambar 4.6 Tampilan aplikasi saat robot mengeksekusi rencana A* pada S3. .... 28
Gambar 4.7 Empat tab panel kanan: PEAS, Masalah, Metrik, dan Rencana. ..... 28
Gambar 4.8 Empat dari lima mode kamera: overview, ikuti, FPV, dan sinema. .. 28
Gambar 4.9 Jendela Lab: perbandingan algoritma pada S1–S4. ..... 29
Gambar 4.10 Mode Editor: menambah kotoran dan perabot dengan klik lantai. ..29
Gambar 4.11 Mode Game “Kamu vs AI”: hasil dibandingkan dengan rencana optimal A*. ..... 30
Gambar 4.12 Jumlah state yang diperiksa tiap algoritma (skala logaritmik). ..... 32
Gambar 4.13 Waktu komputasi pencarian tiap algoritma. ..... 33
Gambar 4.14 Total biaya rencana: A* dibanding Greedy dan Nearest-Neighbor. 33
Gambar 4.15 Heat-map ekspansi state pada S4: BFS (kiri) dan A* MST (kanan).35
Gambar 4.16 Rute optimal A* (pita oranye) pada S1–S4 dalam tampilan 3D. .....35
Gambar 4.17 Rata-rata state diperiksa terhadap jumlah target. .....36
Gambar 4.18 Rata-rata waktu komputasi terhadap jumlah target. ..... 37

# DAFTAR TABEL

Tabel 2.1 Perbandingan teoretis algoritma pencarian yang digunakan. ..... 10
Tabel 3.1 Rumusan PEAS agen pembersih ruang kelas. ..... 14
Tabel 3.2 Lima komponen formulasi masalah pembersihan multi-target. ..... 15
Tabel 3.3 Spesifikasi skenario pengujian. ..... 16
Tabel 3.4 Konfigurasi algoritma yang dibandingkan. ..... 17
Tabel 3.5 Metrik evaluasi. ..... 17
Tabel 4.1 Lingkungan implementasi. ..... 21
Tabel 4.2 Modul pada paket cleanbot. ..... 21
Tabel 4.3 Statistik aset 3D hasil ekspor glTF. ..... 23
Tabel 4.4 Preset kualitas render pada aplikasi. ..... 26
Tabel 4.5 Ringkasan pengujian otomatis. ..... 30
Tabel 4.6 Total biaya rencana (jumlah aksi) pada tiap skenario. ..... 31
Tabel 4.7 Jumlah state yang diperiksa (diperluas) pada tiap skenario. ..... 32
Tabel 4.8 Waktu komputasi pencarian (ms, median 5 kali jalan). ..... 33
Tabel 4.9 Nilai heuristik pada state awal dan jumlah state yang diperiksa A*. .... 34
Tabel 4.10 Hasil uji skalabilitas pada denah acak 10 × 7. ..... 36
Tabel 4.11 Hasil uji penyusunan ulang rencana dengan rintangan tak terduga. .... 37
Tabel A.1 Tombol kontrol aplikasi. ..... 43
Tabel C.1 Hasil benchmark seluruh konfigurasi pada skenario S1–S4. ..... 45

# ABSTRAK

Perencanaan rute merupakan kemampuan inti robot pembersih yang harus mengunjungi beberapa titik kotoran lalu kembali ke titik awal tanpa menabrak rintangan. Laporan ini menyajikan perancangan, implementasi, dan evaluasi agen pembersih ruang kelas yang memodelkan tugasnya sebagai masalah pencarian (*problem solving*). Tugas dirumuskan melalui PEAS dan lima komponen formulasi masalah dengan state s = (posisi robot, himpunan target tersisa). Algoritma *Breadth-First Search* (BFS), *Uniform-Cost Search* (UCS), dan A* dengan tiga heuristik (Manhattan, jarak-max, dan MST + terdekat) dibandingkan, bersama *Greedy Best-First* dan *Nearest-Neighbor* sebagai pembanding non-optimal. Pengujian dilakukan pada tiga skenario denah 8×5 yang dirancang pada UTS, satu skenario tambahan 10×7, dan uji skalabilitas pada denah acak dengan 2–9 target.

Seluruh algoritma optimal menghasilkan biaya yang sama (23, 21, 31, 38 aksi pada S1–S4), tetapi A* dengan heuristik MST memeriksa 4,4 hingga 33,9 kali lebih sedikit state daripada BFS, dan 62 kali lebih sedikit pada denah dengan sembilan target. Greedy dan Nearest-Neighbor menghasilkan biaya 42 pada S4, yaitu 10,5% lebih mahal dari rencana optimal. Rencana dari inti AI Python divisualkan sebagai simulasi 3D yang aset-nya dibuat dengan Blender (bpy) dan dirender secara *real-time* dengan three.js, lengkap dengan visualisasi proses pencarian, mode editor, dan permainan “Kamu vs AI”. Seluruh 20 pengujian otomatis dinyatakan lulus.

**Kata kunci:** agen pemecah masalah, PEAS, A*, heuristik admissible, perencanaan rute, robot pembersih, Blender, three.js.

# ABSTRACT

*Route planning is a core capability of cleaning robots that must visit several dirt locations and return to the start without hitting obstacles. This report presents the design, implementation, and evaluation of a classroom-cleaning agent that models its task as a search problem. The task is formulated with PEAS and the five components of problem formulation, using the state s = (robot position, set of remaining targets). Breadth-First Search (BFS), Uniform-Cost Search (UCS), and A* with three heuristics (Manhattan, max-distance, and MST + nearest) are compared, together with Greedy Best-First and Nearest-Neighbor as non-optimal baselines. Experiments cover three 8×5 layouts designed in the midterm, one additional 10×7 layout, and a scalability test on random layouts with 2–9 targets.*

*All optimal algorithms return identical costs (23, 21, 31, 38 actions for S1–S4), but A* with the MST heuristic expands 4.4 to 33.9 times fewer states than BFS (62 times fewer with nine targets), while Greedy and Nearest-Neighbor are 10.5% more expensive on S4. The plans computed by the Python AI core are visualized in a realistic 3D simulation whose assets are built with Blender (bpy) and rendered in real time with three.js, including search visualization, a map editor, and a "You vs AI" game. All 20 automated tests pass.*

**Keywords:** problem-solving agent, PEAS, A*, admissible heuristic, route planning, cleaning robot, Blender, three.js.

# BAB I
# PENDAHULUAN

## 1.1 Latar Belakang

Robot pembersih (*cleaning robot*) semakin banyak digunakan di lingkungan dalam ruangan seperti rumah, perkantoran, rumah sakit, dan sekolah. Salah satu kemampuan intinya adalah perencanaan rute (*path planning*), yaitu menentukan urutan lokasi yang dikunjungi dan jalur yang ditempuh agar pekerjaan selesai dengan biaya perjalanan sekecil mungkin tanpa menabrak rintangan (Moon et al., 2022).

Ruang kelas adalah contoh lingkungan yang sederhana tetapi representatif. Setelah kegiatan belajar, sampah tersebar pada beberapa titik, sedangkan meja, kursi, dan lemari membatasi gerak robot. Robot harus memutuskan titik mana yang dibersihkan lebih dahulu, jalur mana yang dilalui, dan kembali ke titik awal setelah selesai. Karena urutan kunjungan menentukan biaya total, masalah ini serupa dengan *Traveling Salesman Problem* (TSP) pada jarak terpendek antar-titik, sehingga jumlah urutan yang mungkin tumbuh secara faktorial terhadap banyaknya titik kotoran.

Pada mata kuliah Kecerdasan Buatan, situasi seperti ini dipelajari melalui konsep agen pemecah masalah (*problem-solving agent*), yaitu agen yang merumuskan tujuan, merumuskan masalah, lalu mencari urutan aksi menuju tujuan (Russell & Norvig, 2021). Pada UTS, kelompok kami merancang agen pembersih ruang kelas berdasarkan PEAS dan lima komponen formulasi masalah, serta tiga skenario denah yang akan diuji dengan algoritma A*, dengan BFS atau UCS sebagai pembanding.

Laporan UAS ini melanjutkan rancangan tersebut menjadi proyek yang utuh: rancangan diimplementasikan dalam bahasa Python, dievaluasi secara kuantitatif, dan hasil rencananya divisualkan sebagai simulasi tiga dimensi yang realistis. Visualisasi dipilih karena rencana berupa deretan aksi sulit dipahami hanya dari angka. Dengan animasi robot, peta panas (*heat-map*) ekspansi state, serta antarmuka yang menampilkan nilai g(n), h(n), dan f(n), proses berpikir agen dapat

dijelaskan secara intuitif. Aset tiga dimensi dibuat secara terprogram dengan Blender (Python API *bpy*) dan ditampilkan secara *real-time* dengan pustaka three.js, sedangkan perhitungan rencana tetap dilakukan oleh inti AI Python yang dihubungkan melalui antarmuka REST (FastAPI).

## 1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah pada proyek ini adalah sebagai berikut.

1. Bagaimana memodelkan tugas pembersihan ruang kelas oleh robot sebagai masalah *problem solving* menggunakan PEAS dan lima komponen formulasi masalah?
2. Bagaimana perbandingan BFS, UCS, dan A* (dengan beberapa heuristik) dari sisi optimalitas rencana, jumlah state yang diperiksa, dan waktu komputasi?
3. Bagaimana pengaruh jumlah target dan kualitas heuristik terhadap efisiensi pencarian?
4. Bagaimana merealisasikan rencana tersebut menjadi simulasi 3D yang realistis dan interaktif sehingga proses berpikir agen dapat dijelaskan secara visual?

## 1.3 Tujuan

1. Memodelkan tugas robot pembersih ruang kelas melalui PEAS dan lima komponen formulasi masalah.
2. Menghasilkan rencana perjalanan yang mengunjungi seluruh target dan kembali ke titik awal, dengan BFS, UCS, dan A* sebagai algoritma utama.
3. Mengevaluasi keberhasilan tugas, total biaya, jumlah state yang diperiksa, dan waktu komputasi pada beberapa skenario denah serta uji skalabilitas.
4. Membangun simulasi 3D yang realistis dan interaktif dengan memanfaatkan Python, Blender, dan three.js untuk memvisualkan proses pencarian dan eksekusi rencana.

## 1.4 Manfaat

Secara teoretis, proyek ini memperlihatkan penerapan konsep agen, formulasi masalah, pencarian tak-berinformasi dan berinformasi, serta perancangan heuristik *admissible* dan *consistent* pada kasus nyata yang memiliki banyak target. Secara praktis, sistem yang dihasilkan dapat dijadikan alat peraga pembelajaran pencarian AI serta kerangka dasar perencanaan rute robot pembersih. Simulasi 3D-nya dapat dipakai untuk mencoba denah baru sebelum diterapkan pada robot sungguhan.

## 1.5 Batasan Masalah

1. Lingkungan dimodelkan sebagai grid dua dimensi yang diskret; satu sel setara 1 m × 1 m dan robot bergerak ke empat arah (atas, kanan, bawah, kiri).
2. Titik kotoran dan denah perabot diketahui di awal (lingkungan teramati penuh dan statis). Pengecualian hanya pada uji penyusunan ulang rencana (*replanning*) ketika rintangan baru muncul.
3. Setiap aksi, baik bergerak maupun membersihkan, berbiaya 1 satuan; nilai ini adalah asumsi simulasi, bukan pengukuran energi robot sungguhan.
4. Sistem terdiri dari satu robot; koordinasi beberapa robot dan pengelolaan baterai tidak dibahas.
5. Proyek berupa simulasi perangkat lunak. Dinamika fisik robot (percepatan, kemiringan bodi, putaran roda) hanya dianimasikan untuk keperluan visual dan tidak memengaruhi hasil perencanaan.
6. Algoritma yang dibandingkan terbatas pada BFS, UCS, A* (tiga heuristik), Greedy Best-First, dan Nearest-Neighbor.

## 1.6 Sistematika Penulisan

Laporan ini disusun dalam lima bab. Bab I Pendahuluan menguraikan latar belakang, rumusan masalah, tujuan, manfaat, dan batasan. Bab II Landasan Teori membahas agen cerdas, agen pemecah masalah, algoritma pencarian, fungsi heuristik, dan teknologi pendukung. Bab III Metodologi dan Perancangan menjelaskan tahapan pengembangan, analisis PEAS, formulasi masalah, skenario pengujian, metrik evaluasi, arsitektur sistem, dan perancangan simulasi 3D. Bab

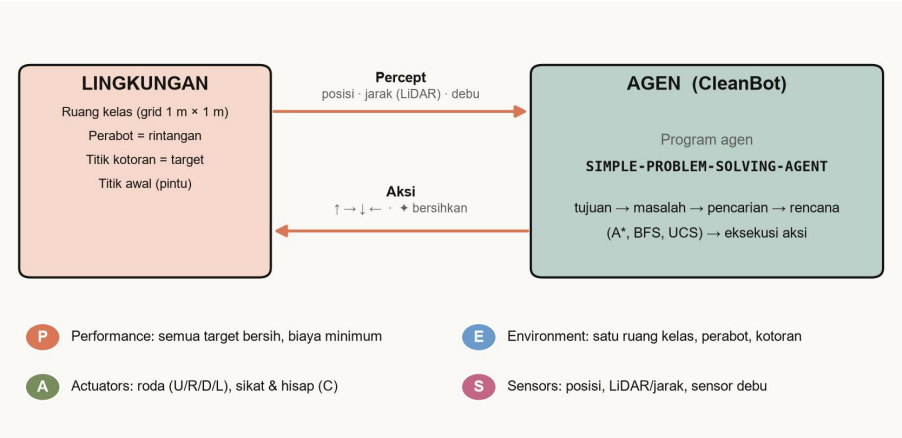
IV Implementasi, Hasil, dan Pembahasan menyajikan realisasi sistem, pengujian, hasil eksperimen, dan analisisnya. Bab V Penutup memuat kesimpulan dan saran. Lampiran memuat petunjuk menjalankan aplikasi, struktur proyek, hasil benchmark lengkap, dan cuplikan kode inti.

# BAB II
# LANDASAN TEORI

## 2.1 Agen Cerdas dan Lingkungan

### *2.1.1 Konsep Agen*

Agen adalah segala sesuatu yang mempersepsi lingkungannya melalui sensor dan bertindak terhadap lingkungan tersebut melalui aktuator (Russell & Norvig, 2021). Masukan sesaat dari sensor disebut *percept*, sedangkan seluruh riwayat percept disebut urutan percept (*percept sequence*). Fungsi agen memetakan urutan percept ke aksi, dan program agen adalah implementasi konkret fungsi tersebut. Agen disebut rasional apabila untuk setiap urutan percept ia memilih aksi yang diharapkan memaksimalkan ukuran performa, berdasarkan bukti dari percept dan pengetahuan bawaan yang dimilikinya.




**Gambar 2.1** Interaksi agen dengan lingkungan dan ringkasan PEAS.

### *2.1.2 PEAS*

Lingkungan tugas (*task environment*) ditentukan dengan PEAS: *Performance measure* (ukuran keberhasilan), *Environment* (lingkungan), *Actuators* (aktuator), dan *Sensors* (sensor). Perumusan PEAS menjadi langkah pertama dalam merancang agen karena mengharuskan perancang menyebutkan secara eksplisit apa yang dianggap berhasil, apa yang dapat dipersepsi, dan apa yang dapat dilakukan agen. Penerapan PEAS pada robot pembersih ruang kelas diuraikan pada Bab III.

### *2.1.3 Sifat Lingkungan Tugas*

Lingkungan tugas dapat dikarakterisasi berdasarkan beberapa dimensi: teramati penuh atau sebagian, deterministik atau stokastik, episodik atau sekuensial, statis atau dinamis, diskret atau kontinu, serta agen tunggal atau jamak. Pada proyek ini lingkungan bersifat teramati penuh (denah dan lokasi kotoran diketahui), deterministik (aksi selalu menghasilkan efek yang sama), sekuensial (aksi sekarang memengaruhi biaya selanjutnya), statis (kecuali pada uji penyusunan ulang rencana), diskret (grid), dan melibatkan agen tunggal. Kombinasi sifat ini memungkinkan agen menyusun seluruh rencana di awal melalui pencarian dan mengeksekusinya tanpa umpan balik, sambil tetap siap menyusun ulang rencana apabila percept menunjukkan hal tak terduga.

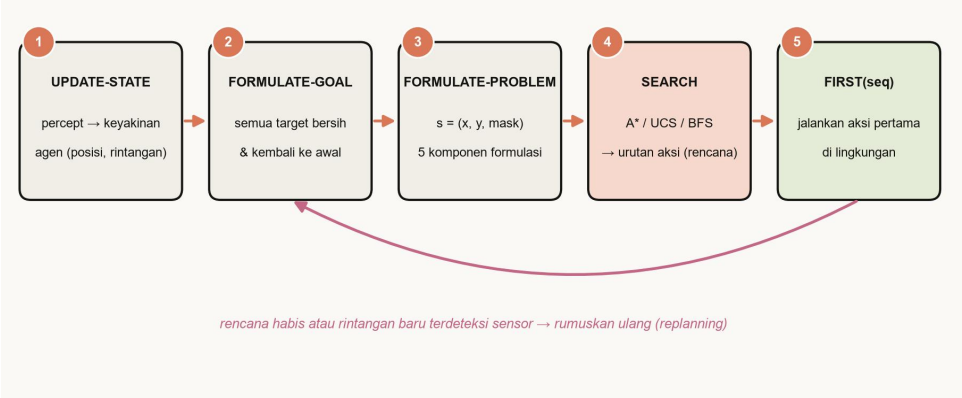
### *2.1.4 Jenis Agen*

Berdasarkan struktur program, agen dibedakan menjadi agen refleks sederhana, agen refleks berbasis model, agen berbasis tujuan (*goal-based*), agen berbasis utilitas, dan agen pembelajar. Agen pembersih pada proyek ini tergolong agen berbasis tujuan: ia menyimpan model lingkungan (denah dan daftar target), merumuskan tujuan eksplisit, lalu memilih aksi melalui pencarian urutan aksi yang mencapai tujuan tersebut.

## 2.2 Agen Pemecah Masalah

### *2.2.1 Siklus Agen*

Agen pemecah masalah bekerja dalam empat fase. Pertama, *perumusan tujuan* (*goal formulation*), yaitu membatasi perilaku agen pada kumpulan state tujuan. Kedua, *perumusan masalah* (*problem formulation*), yaitu memutuskan aksi dan state apa yang dipertimbangkan. Ketiga, *pencarian* (*search*), yaitu mensimulasikan urutan aksi pada model hingga ditemukan urutan yang mencapai tujuan. Keempat, *eksekusi*, yaitu menjalankan aksi pada rencana. Siklus ini dirangkum pada Gambar 2.2 dan diterjemahkan menjadi program agen pada Kode 2.1.




**Gambar 2.2** Siklus agen pemecah masalah beserta penyusunan ulang rencana.

**Kode 2.1** Program agen pemecah masalah sederhana (Russell & Norvig, 2021).

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
    persistent: seq, an action sequence, initially empty
                state, some description of the current world state
                goal, a goal, initially null
                problem, a problem formulation
    state <- UPDATE-STATE(state, percept)
    if seq is empty then
        goal    <- FORMULATE-GOAL(state)
        problem <- FORMULATE-PROBLEM(state, goal)
        seq     <- SEARCH(problem)
        if seq = failure then return a null action
    action <- FIRST(seq)
    seq    <- REST(seq)
    return action
```

### *2.2.2 Formulasi Masalah*

Masalah pencarian dirumuskan secara formal melalui lima komponen berikut (Russell & Norvig, 2021).

1. **Keadaan awal** (*initial state*): keadaan tempat agen memulai.
2. **Aksi** (*actions*): himpunan aksi yang dapat dilakukan pada suatu state s, dinotasikan ACTIONS(s).
3. **Model transisi** (*transition model*): deskripsi efek aksi, dinotasikan RESULT(s, a) yang menghasilkan state berikutnya.
4. **Uji tujuan** (*goal test*): fungsi yang menentukan apakah suatu state adalah state tujuan.
5. **Biaya jalur** (*path cost*): fungsi yang memberi biaya numerik pada jalur; biaya langkah c(s, a, s′) dijumlahkan sepanjang jalur.

Solusi adalah urutan aksi dari keadaan awal hingga state tujuan, dan solusi optimal adalah solusi dengan biaya jalur terendah. Keempat komponen pertama mendefinisikan ruang state (*state space*) berupa graf yang simpulnya adalah state

dan sisinya adalah aksi. Pencarian menelusuri graf tersebut dengan menjaga batas (*frontier*) berisi simpul yang telah dibangkitkan namun belum diperluas, serta himpunan state yang telah diperluas (*explored set* atau *closed set*) agar state yang sama tidak diproses berulang.

## 2.3 Algoritma Pencarian

### *2.3.1 Kriteria Penilaian*

Algoritma pencarian dinilai dengan empat kriteria: *kelengkapan* (apakah selalu menemukan solusi bila ada), *optimalitas* (apakah solusi yang ditemukan berbiaya terendah), *kompleksitas waktu*, dan *kompleksitas ruang*. Kompleksitas biasanya dinyatakan dengan faktor percabangan b (jumlah suksesor per simpul), kedalaman solusi terdangkal d, dan kedalaman maksimum ruang state m.

### *2.3.2 Breadth-First Search (BFS)*

BFS memperluas simpul terdangkal lebih dahulu dengan antrian FIFO. Algoritma ini lengkap bila b berhingga dan optimal bila semua aksi berbiaya sama, karena solusi terdangkal pasti berbiaya terendah. Kompleksitas waktu dan ruangnya $O(b^d)$, sehingga memori menjadi kendala utama. Pada implementasi kami, uji tujuan dilakukan saat state dibangkitkan (*early goal test*), mengikuti Russell dan Norvig (2021).

### *2.3.3 Uniform-Cost Search (UCS)*

UCS (varian algoritma Dijkstra yang berhenti saat tujuan ditemukan; Dijkstra, 1959) memperluas simpul dengan biaya jalur g(n) terkecil menggunakan antrian prioritas. UCS lengkap dan optimal selama setiap biaya langkah lebih besar dari konstanta $\varepsilon > 0$. Pada masalah dengan biaya langkah seragam seperti proyek ini, UCS berperilaku hampir sama dengan BFS; perbedaannya hanya pada saat uji tujuan, yaitu ketika simpul diperluas, bukan ketika dibangkitkan.

### *2.3.4 Algoritma A**

A* memperluas simpul dengan nilai f(n) terkecil, dengan f(n) menjumlahkan biaya yang sudah ditempuh dan perkiraan biaya yang tersisa (Hart et al., 1968):

$$f(n) = g(n) + h(n) \tag{2.1}$$

Di sini g(n) adalah biaya jalur dari keadaan awal ke simpul n dan h(n) adalah perkiraan biaya terendah dari n ke tujuan. A* optimal bila h admissible pada pencarian pohon, dan bila h consistent pada pencarian graf (Dechter & Pearl, 1985). Dibandingkan UCS, A* hanya memperluas simpul yang tampak menjanjikan menurut h(n), sehingga umumnya memeriksa jauh lebih sedikit state. Pada implementasi kami, simpul dengan nilai f sama diurutkan menurut h yang lebih kecil (lebih dekat ke tujuan), dan uji tujuan dilakukan saat simpul akan diperluas.

### *2.3.5 Greedy Best-First Search dan Nearest-Neighbor*

Greedy Best-First Search hanya memakai h(n) untuk memilih simpul. Algoritma ini cepat, tetapi tidak menjamin solusi optimal karena mengabaikan biaya yang sudah ditempuh. Nearest-Neighbor adalah heuristik rakus klasik untuk TSP: dari posisi sekarang, robot selalu menuju target terdekat yang belum dibersihkan, lalu kembali ke titik awal setelah semua target selesai. Nearest-Neighbor bukan pencarian pada ruang state, sehingga tidak menghasilkan hitungan state yang diperiksa. Keduanya dipakai sebagai pembanding untuk memperlihatkan bahwa urutan kunjungan menentukan biaya.

**Tabel 2.1** Perbandingan teoretis algoritma pencarian yang digunakan.

| Algoritma | Lengkap | Optimal | Waktu | Ruang |
|---|---|---|---|---|
| BFS | Ya (b berhingga) | Ya, bila biaya langkah seragam | $O(b^d)$ | $O(b^d)$ |
| UCS | Ya (biaya $\geq \varepsilon > 0$) | Ya | $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ | $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ |
| A* | Ya (biaya $\geq \varepsilon > 0$) | Ya, bila h admissible (pohon) / consistent (graf) | Bergantung pada h; eksponensial pada kasus terburuk | Seluruh simpul disimpan |
| Greedy | Tidak pada pencarian pohon; ya pada graf berhingga | Tidak | $O(b^m)$ kasus terburuk | $O(b^m)$ |
| Nearest-Neighbor | Ya, bila semua target terjangkau | Tidak | Polinomial (k pencarian BFS) | O(jumlah sel) |

*Keterangan: b = faktor percabangan, d = kedalaman solusi terdangkal, m = kedalaman maksimum, C* = biaya solusi optimal, k = jumlah target.*

## 2.4 Fungsi Heuristik

### *2.4.1 Admissible dan Consistent*

Heuristik h(n) disebut *admissible* bila tidak pernah melebihi biaya sebenarnya untuk mencapai tujuan, yaitu $h(n) \leq h^*(n)$ untuk setiap simpul n. Heuristik disebut *consistent* bila untuk setiap simpul n dan suksesor n′ yang dihasilkan aksi a berlaku ketidaksamaan segitiga:

$$h(n) \leq c(n, a, n') + h(n') \tag{2.2}$$

Heuristik yang consistent selalu admissible, dan nilai f tidak pernah menurun sepanjang jalur. Akibatnya A* pada pencarian graf dengan heuristik consistent tidak perlu membuka kembali state yang sudah diperluas, dan solusi pertama yang ditemukan sudah optimal. Heuristik yang lebih besar namun tetap admissible dikatakan *mendominasi* heuristik yang lebih kecil, dan umumnya membuat A* memeriksa lebih sedikit state.

### *2.4.2 Heuristik untuk Pembersihan Multi-Target*

Pada masalah pembersihan multi-target, sisa pekerjaan dari suatu state adalah mengunjungi seluruh target tersisa R lalu kembali ke titik awal. Karena

setiap target membutuhkan tepat satu aksi "bersihkan", suku |R| dapat ditambahkan pada semua heuristik. Dengan d(a, b) sebagai jarak terpendek pada grid (dihitung dengan BFS) dan home sebagai titik awal, tiga heuristik yang dipakai pada proyek ini adalah sebagai berikut.

$$h_M(s) = max_{t \in R} [\ |pos - t| + |t - home|\ ]_{Manhattan} + |R| \quad (2.3)$$

$$h_{max}(s) = max_{t \in R} [\ d(pos, t) + d(t, home)\ ] + |R| \quad (2.4)$$

$$h_{MST}(s) = MST(R \cup \{home\}) + min_{u \in R \cup \{home\}} d(pos, u) + |R| \quad (2.5)$$

Heuristik $h_M$ adalah versi paling sederhana karena mengabaikan rintangan. Heuristik $h_{max}$ memakai jarak sebenarnya sehingga lebih ketat. Heuristik $h_{MST}$ memanfaatkan *Minimum Spanning Tree* (MST) yang dihitung dengan algoritma Prim (Prim, 1957) atas simpul target tersisa dan titik awal dengan bobot sisi berupa jarak BFS. Bobot MST adalah batas bawah panjang jalan apa pun yang menyinggung semua simpul tersebut, ditambah jarak terdekat dari posisi robot ke salah satu simpul. Dengan demikian $h_{MST}$ tidak pernah melebihi biaya sisa sebenarnya sehingga admissible. Konsistensinya mengikuti dua fakta: jarak terdekat berubah paling banyak 1 untuk setiap gerakan, dan menambahkan satu simpul ke himpunan menaikkan bobot MST paling banyak sebesar jarak simpul itu ke simpul terdekat lainnya. Ketiga heuristik juga diverifikasi secara empiris melalui pengujian otomatis (Bab IV).

Gagasan merepresentasikan state sebagai pasangan posisi dan himpunan target yang belum dikunjungi sejalan dengan pemrograman dinamis untuk TSP oleh Held dan Karp (1962), yang juga membedakan state berdasarkan subhimpunan simpul yang telah dikunjungi. Konsekuensinya, ukuran ruang state berada pada orde (jumlah sel bebas) $\times 2^k$ untuk k target.

## 2.5 Teknologi Pendukung

### *2.5.1 Python, FastAPI, dan Pustaka Analisis*

Python dipilih sebagai bahasa implementasi inti AI sesuai ketentuan tugas. Antarmuka antara inti AI dan aplikasi 3D dibangun dengan FastAPI (dijalankan oleh server ASGI uvicorn) yang menyediakan titik akhir REST berformat JSON. NumPy dan Pillow dipakai untuk membangkitkan tekstur prosedural, Matplotlib

untuk grafik laporan, pytest untuk pengujian otomatis, serta Playwright untuk menangkap layar aplikasi 3D secara otomatis dengan akselerasi GPU.

### *2.5.2 Blender dan bpy*

Blender adalah perangkat lunak pemodelan 3D sumber terbuka yang seluruh fungsinya dapat dikendalikan melalui Python API (*bpy*) dan dijalankan tanpa antarmuka (*headless*). Model dibangun dari primitif mesh melalui modul *bmesh* dengan bevel agar tepi menangkap cahaya secara realistis. Material memakai Principled BSDF yang berbasis model Disney (Burley, 2012) dengan parameter warna dasar, kekasaran (*roughness*), kilap logam (*metallic*), lapisan pelindung (*clearcoat*), dan emisi. Model diekspor ke format glTF 2.0 (berkas .glb) yang didukung langsung oleh three.js. Mesin Cycles dengan akselerasi OptiX digunakan untuk render fotorealistis.

### *2.5.3 three.js*

three.js adalah pustaka JavaScript untuk menampilkan grafika 3D di peramban berbasis WebGL. Fitur yang dimanfaatkan pada proyek ini meliputi material fisik (*MeshPhysicalMaterial*), pencahayaan berbasis gambar (*image-based lighting*) dari lingkungan prosedural, peta bayangan (*shadow map*) dari cahaya matahari, *GLTFLoader* untuk memuat aset Blender, dan rantai pascapemrosesan (*EffectComposer*) berisi *ambient occlusion* (GTAO) dan *bloom*.

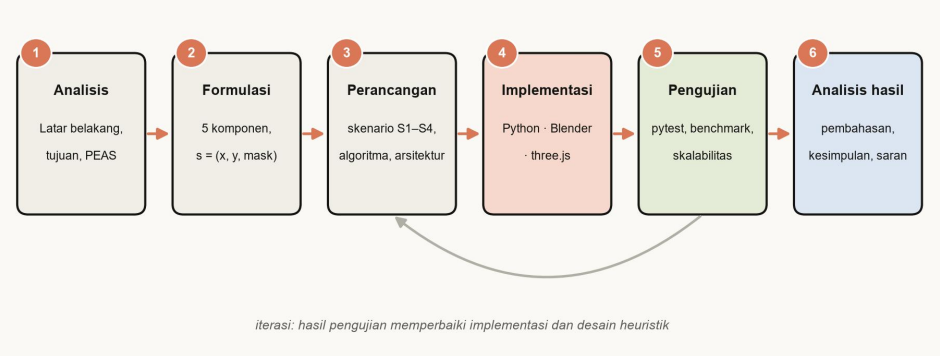
## 2.6 Penelitian Terkait

Moon et al. (2022) mengusulkan perencanaan jalur robot pembersih berbasis *reinforcement learning*. Pendekatan pembelajaran berguna ketika lingkungan tidak diketahui, tetapi memerlukan proses pelatihan dan tidak menjamin optimalitas. Sebaliknya, pendekatan berbasis pencarian yang dipakai pada proyek ini memanfaatkan denah yang diketahui untuk menghasilkan rencana yang terbukti optimal dan dapat dijelaskan langkah demi langkah, sehingga cocok untuk tujuan pembelajaran dan visualisasi. Rujukan NIST tentang kinerja mobilitas sistem robotik menegaskan perlunya metrik yang terukur dan dapat diulang, sejalan dengan metrik evaluasi pada proyek ini.

# BAB III
# METODOLOGI DAN PERANCANGAN

## 3.1 Tahapan Pengembangan

Pengembangan dilakukan dalam enam tahap yang digambarkan pada Gambar 3.1. Tahap analisis dan formulasi sudah dikerjakan pada UTS; tahap perancangan hingga analisis hasil dikerjakan pada UAS.




**Gambar 3.1** Tahapan pengembangan proyek.

1. **Analisis**: menentukan latar belakang, tujuan, dan karakteristik tugas melalui PEAS.
2. **Formulasi**: merumuskan state, aksi, model transisi, uji tujuan, dan biaya sebagai masalah pencarian.
3. **Perancangan**: menyusun skenario pengujian, memilih algoritma dan heuristik, serta merancang arsitektur sistem dan simulasi 3D.
4. **Implementasi**: membangun inti AI di Python, aset 3D di Blender, dan simulasi real-time di three.js.
5. **Pengujian**: menjalankan pengujian otomatis, benchmark pada S1–S4, dan uji skalabilitas pada denah acak.
6. **Analisis hasil**: membahas hasil, menarik kesimpulan, dan merumuskan saran pengembangan.

## 3.2 Analisis Tugas dengan PEAS

**Tabel 3.1** Rumusan PEAS agen pembersih ruang kelas.

| Komponen | Uraian |
|---|---|
| **Performance** | Seluruh target kotoran dibersihkan, robot tidak menabrak perabot (nol tabrakan), robot kembali ke titik awal, dan total biaya perjalanan sekecil mungkin. |
| **Environment** | Satu ruang kelas berbentuk grid; perabot (meja, lemari, rak, meja guru, tempat sampah) berfungsi sebagai rintangan; terdapat titik kotoran sebagai target dan titik awal di dekat pintu. |
| **Actuators** | Roda penggerak untuk bergerak ke atas, kanan, bawah, dan kiri; sikat dan penghisap untuk membersihkan. |
| **Sensors** | Posisi robot, sensor jarak/LiDAR (apakah sel tetangga terhalang), sensor debu (apakah ada kotoran pada sel saat ini), serta data denah ruangan. |

PEAS tersebut diimplementasikan secara langsung pada kelas ClassroomEnvironment dan CleaningAgent di modul `agent.py`. Percept berisi nomor langkah, posisi, status sensor debu, dan status terhalang pada keempat arah. Ukuran performa dihitung oleh lingkungan: jumlah gerakan, jumlah pembersihan, total biaya, dan jumlah tabrakan.

## 3.3 Formulasi Masalah

### *3.3.1 Representasi State*

State dinyatakan sebagai s = (x, y, mask), dengan (x, y) posisi robot pada grid dan mask sebuah bitmask yang bit ke-i bernilai 1 bila target ke-i belum dibersihkan. Representasi ini ringkas dan memungkinkan penghapusan target dengan satu operasi bit. Dalam kode, state diwakili oleh `State(NamedTuple)` sehingga dapat dijadikan kunci kamus. Batas atas jumlah state adalah (jumlah sel bebas) $\times 2^k$.

### *3.3.2 Lima Komponen Formulasi*

**Tabel 3.2** Lima komponen formulasi masalah pembersihan multi-target.

| Komponen | Definisi | Implementasi (kelas CleaningProblem) |
|---|---|---|
| **Keadaan awal** | Robot berada di titik awal dan semua target belum dibersihkan: s_0 = (x_0, y_0, 2^k − 1). | `initial_state()` |
| **Aksi** | Bergerak (U, R, D, L) ke sel tetangga yang berada di dalam ruangan dan bukan rintangan; membersihkan (C) bila posisi saat ini adalah target yang masih kotor. | `actions(s)`, `successors(s)` |
| **Model transisi** | Gerakan mengubah (x, y); aksi C menghapus bit target pada mask; aksi lain tidak mengubah mask. | `result(s, a)` |
| **Uji tujuan** | mask = 0 dan posisi robot sama dengan titik awal. | `goal_test(s)` |
| **Biaya jalur** | Biaya langkah 1 untuk setiap aksi (gerak maupun bersihkan); biaya jalur adalah jumlah langkah. | `step_cost(s, a)` |

Urutan aksi dibuat tetap (C, U, R, D, L) agar hasil pencarian deterministik dan dapat direproduksi. Target yang berada di atas rintangan atau tidak terjangkau dari titik awal dideteksi sebelum pencarian dan dilaporkan sebagai target yang dilewati, sehingga agen tidak mencari tanpa akhir.

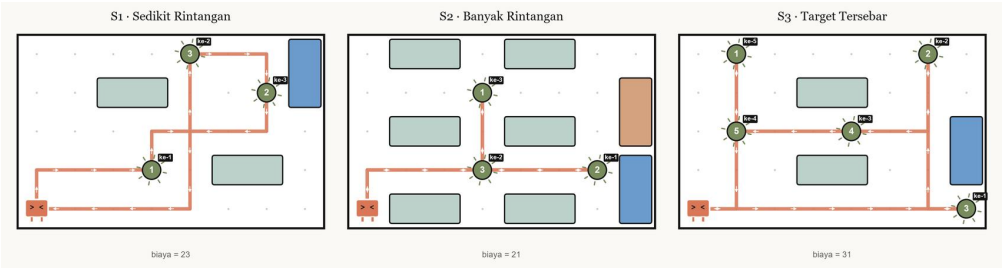
## 3.4 Skenario Pengujian

Pengujian memakai empat skenario denah. Skenario S1, S2, dan S3 sama persis dengan rancangan UTS: grid 8 kolom × 5 baris dengan robot berangkat dari pintu di pojok kiri-bawah. S1 memiliki sedikit rintangan, S2 memiliki banyak rintangan dengan koridor sempit, dan S3 memiliki lima target yang tersebar di berbagai sudut. Skenario S4 adalah denah tambahan berukuran 10 × 7 berisi enam meja panjang, meja guru, lemari, rak buku, tempat sampah, dan enam target; S4 dipakai untuk menguji skalabilitas sekaligus sebagai tampilan utama simulasi. Spesifikasi lengkap diberikan pada Tabel 3.3, dan denah beserta rute optimal pada Gambar 3.2 dan Gambar 3.3. Pada gambar denah, kotak hijau menandakan meja, kotak biru lemari, kotak cokelat rak buku, dan angka menandakan urutan pembersihan pada rute optimal.

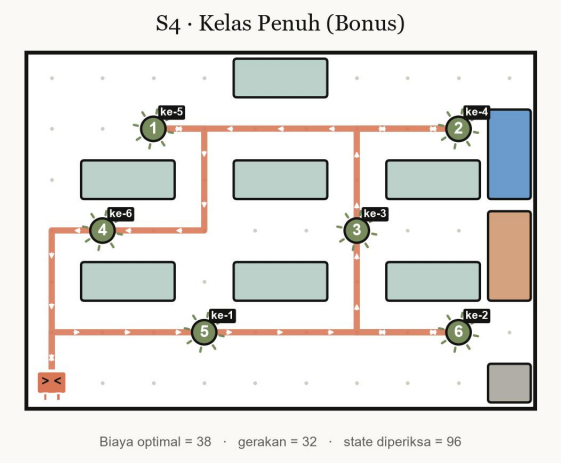
**Tabel 3.3** Spesifikasi skenario pengujian.

| ID | Nama | Grid | Sel bebas | k | Koordinat target (x, y) | Ruang state |
|---|---|---|---|---|---|---|
| S1 | Sedikit Rintangan | 8×5 | 34 | 3 | 1:(3,3) 2:(6,1) 3:(4,0) | 272 |
| S2 | Banyak Rintangan | 8×5 | 24 | 3 | 1:(3,1) 2:(6,3) 3:(3,3) | 192 |
| S3 | Target Tersebar | 8×5 | 34 | 5 | 1:(1,0) 2:(6,0) 3:(7,4) 4:(4,2) 5:(1,2) | 1.088 |
| S4 | Kelas Penuh (Bonus) | 10×7 | 51 | 6 | 1:(2,1) 2:(8,1) 3:(6,3) 4:(1,3) 5:(3,5) 6:(8,5) | 3.264 |

*Keterangan: titik awal berada di (0, 4) untuk S1–S3 dan (0, 6) untuk S4; x = kolom (0 di sisi pintu), y = baris (0 di dekat papan tulis).*




**Gambar 3.2** Denah skenario S1–S3 (rancangan UTS) beserta rute optimal A*.




**Gambar 3.3** Denah skenario S4 (kelas penuh 10 × 7) beserta rute optimal A*.

Selain skenario tetap, dilakukan uji skalabilitas pada denah acak berukuran 10 × 7 dengan jumlah target k = 2 sampai 9. Untuk setiap k dibangkitkan lima denah dengan benih acak 0 sampai 4 (total 40 denah). Generator denah menempatkan meja dan lemari secara acak lalu memilih lokasi target hanya dari sel yang terjangkau, sehingga setiap denah selalu dapat diselesaikan.

## 3.5 Algoritma dan Heuristik yang Diuji

**Tabel 3.4** Konfigurasi algoritma yang dibandingkan.

| Kode | Algoritma | Prioritas / aturan | Jaminan optimal |
|---|---|---|---|
| `bfs` | BFS | Antrian FIFO; uji tujuan saat dibangkitkan | Ya (biaya seragam) |
| `ucs` | UCS | g(n) terkecil | Ya |
| `astar_manhattan` | A* (Manhattan) | $f = g + h_M$ | Ya |
| `astar_max` | A* (jarak-max) | $f = g + h_{max}$ | Ya |
| `astar_mst` | A* (MST) | $f = g + h_{MST}$ | Ya |
| `greedy_mst` | Greedy (MST) | $h_{MST}$ saja | Tidak |
| `nn` | Nearest-Neighbor | Selalu ke target terdekat | Tidak |

Semua algoritma pencarian bekerja pada ruang state yang sama dan memakai himpunan tertutup (*closed set*) agar state tidak diperluas dua kali. Jarak BFS dari setiap target dan dari titik awal dihitung satu kali sebelum pencarian dan disimpan sebagai tabel, sehingga evaluasi h(n) berlangsung dalam waktu hampir konstan; bobot MST untuk setiap himpunan target tersisa disimpan pada cache. Waktu penyiapan heuristik ini termasuk dalam waktu komputasi yang dilaporkan.

## 3.6 Metrik Evaluasi

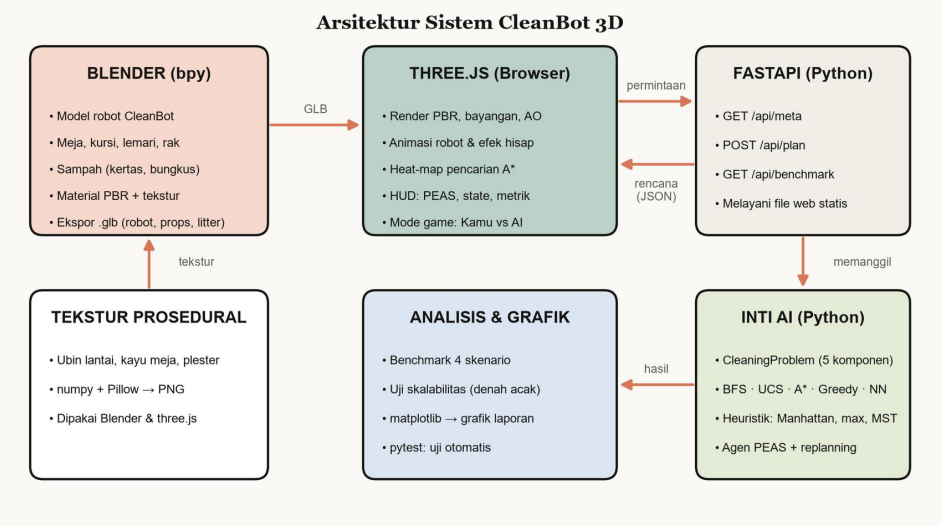
**Tabel 3.5** Metrik evaluasi.

| Metrik | Definisi | Satuan / cara ukur |
|---|---|---|
| **Tugas selesai** | Seluruh target bersih dan robot kembali ke titik awal. | Ya / tidak |
| **Total gerakan** | Jumlah aksi gerak pada rencana. | Langkah |
| **Total biaya** | Jumlah biaya aksi gerak dan bersihkan (1 per aksi). | Satuan biaya |
| **State diperiksa** | Jumlah state yang diperluas oleh algoritma. | State |
| **State dibangkitkan** | Jumlah state yang dimasukkan ke batas pencarian. | State |
| **Waktu komputasi** | Lama pencarian termasuk penyiapan heuristik. | ms, median 5 kali jalan |
| **Selisih biaya** | (biaya − biaya optimal) / biaya optimal. | Persen |
| **Tabrakan** | Jumlah gerakan yang menabrak rintangan selama eksekusi. | Kejadian |

Pada skenario tetap, waktu komputasi adalah median dari lima kali pengulangan untuk meredam fluktuasi pengukuran. Pada uji skalabilitas, hasil tiap k adalah rata-rata dari lima denah acak dengan satu kali jalan per denah.

### 3.7 Arsitektur Sistem

Sistem terdiri atas tiga lapis yang diperlihatkan pada Gambar 3.4. Lapis pertama adalah inti AI Python yang memuat formulasi masalah, algoritma pencarian, heuristik, agen PEAS, benchmark, dan pembuat grafik. Lapis kedua adalah aset 3D yang dibangun secara terprogram dengan Blender dan diekspor sebagai berkas .glb. Lapis ketiga adalah aplikasi web three.js yang menampilkan dunia 3D dan menganimasikan robot berdasarkan rencana dari lapis pertama.




**Gambar 3.4** Arsitektur sistem CleanBot 3D.

Alur data berjalan sebagai berikut. Peramban mengirim permintaan POST ke `/api/plan` berisi spesifikasi denah (ukuran, rintangan, titik awal, target), nama algoritma, dan nama heuristik. Server FastAPI membangun objek `CleaningProblem`, menjalankan `solve()`, lalu mengembalikan JSON yang memuat rangkaian aksi (misalnya "RRDC…"), daftar sel yang dilalui, urutan pembersihan target, nilai h(n) sepanjang rute, statistik pencarian, serta data ekspansi state untuk heat-map. Titik akhir lain adalah `/api/meta` (daftar skenario, algoritma, heuristik) dan `/api/benchmark` (hasil perbandingan untuk jendela Lab). Server juga melayani berkas statis aplikasi web. Bila server Python tidak

aktif, aplikasi memakai data cadangan di folder `web/data` sehingga skenario S1–S4 tetap dapat dijalankan.

## 3.8 Perancangan Simulasi 3D

### *3.8.1 Pemetaan Koordinat dan Aset*

Satu sel grid setara 1 m pada dunia 3D. Kolom grid x dipetakan ke sumbu +X dan baris grid y ke sumbu +Z (ke arah selatan), dengan sumbu Y sebagai tinggi. Pada Blender, koordinat (x, y, z) dipetakan ke (x, z, −y) pada three.js saat ekspor glTF. Depan robot menghadap sumbu +X. Aset dirancang dalam tiga kelompok: robot CleanBot, properti kelas (meja, kursi, lemari, rak buku, meja guru, papan tulis, pintu, jendela, jam, AC, lampu, tempat sampah, tanaman, dok pengisian), dan jenis sampah sebagai penanda target (kertas, bungkus, pensil, kotak jus, remah, tumpahan, debu, penghapus).

### *3.8.2 Alur Sutradara dan Antarmuka*

Modul *Director* mengatur alur agen sebagai mesin keadaan: diam (idle), merumuskan tujuan, merumuskan masalah, mencari, rencana siap, mengeksekusi, selesai. Kelima tahap utama ditampilkan pada antarmuka sebagai penanda tahap di bagian bawah layar agar audiens dapat mengikuti fase agen pemecah masalah secara langsung. Panel kanan menampilkan PEAS yang diperbarui setiap langkah, state saat ini beserta nilai g(n), h(n), dan f(n), metrik pencarian, serta daftar aksi. Panel kiri berisi pilihan skenario, algoritma, heuristik, kecepatan, dan kamera.

### *3.8.3 Visualisasi Pencarian, Kamera, dan Interaksi*

- **Heat-map ekspansi**: setiap sel lantai diwarnai menurut banyaknya state yang diperluas pada sel tersebut sehingga perbedaan BFS dan A* terlihat jelas.
- **Rute menyala**: pita bercahaya di lantai menunjukkan rute rencana; penanda bernomor menunjukkan urutan kunjungan target.
- **Lima mode kamera**: overview, atas, ikuti robot, sudut pandang robot (FPV), dan sinema.
- **Sensor LiDAR**: kipas pindai di lantai dan titik hantam pada perabot memvisualkan sensor jarak.

- **Mode Editor**: pengguna menambah atau menghapus kotoran dan perabot dengan klik lantai, lalu A* menyusun ulang rencana.
- **Mode Game "Kamu vs AI"**: pengguna mengendalikan robot dengan keyboard dan membandingkan biayanya dengan rencana optimal A*.
- **Lab algoritma**: jendela yang membandingkan seluruh algoritma pada S1–S4 dan menampilkan grafik skalabilitas.

# BAB IV
# IMPLEMENTASI, HASIL, DAN PEMBAHASAN

## 4.1 Lingkungan Implementasi

Seluruh pengembangan dan pengujian dilakukan pada satu komputer jinjing dengan spesifikasi dan versi perangkat lunak yang dirangkum pada Tabel 4.1. Versi dicatat agar hasil dapat direproduksi.

**Tabel 4.1** Lingkungan implementasi.

| Komponen | Spesifikasi / versi |
|---|---|
| **Sistem operasi** | Windows 11 Home Single Language (10.0.26200) |
| **GPU** | NVIDIA GeForce RTX 4060 Laptop GPU |
| **Bahasa pemrograman inti AI** | Python 3.13 |
| **Pustaka Python** | FastAPI 0.142, uvicorn 0.53, NumPy 2.3, Matplotlib 3.11, Pillow 12.3, pytest 8.4, Playwright 1.64 |
| **Pemodelan 3D** | Blender 5.2 (Python API bpy), mesin render Cycles dengan OptiX |
| **Simulasi web** | three.js r187 (OrbitControls, GLTFLoader, EffectComposer, GTAOPass, UnrealBloomPass, RoomEnvironment) |
| **Peramban** | Google Chrome (rendering GPU melalui ANGLE/Direct3D 11) |
| **Pembuatan dokumen** | Node.js 24 (pptxgenjs dan docx-js) untuk presentasi dan laporan ini |

## 4.2 Implementasi Inti AI (Python)

Inti AI berada pada paket `cleanbot` yang terdiri atas sepuluh modul fungsional dengan sekitar 1.900 baris kode. Pembagian tanggung jawab modul dirangkum pada Tabel 4.2.

**Tabel 4.2** Modul pada paket cleanbot.

| Modul | Tanggung jawab |
|---|---|
| `scenarios.py` | Denah S1–S4, validasi denah (perabot tidak bertumpuk, target terjangkau), dan generator denah acak. |
| `problem.py` | Kelas CleaningProblem: lima komponen formulasi, tabel tetangga, medan jarak BFS, ukuran ruang state. |
| `heuristics.py` | Heuristik Manhattan, jarak-max, dan MST + terdekat beserta cache. |
| `search.py` | BFS, UCS, A*, Greedy, Nearest-Neighbor, dan perekam ekspansi state untuk heat-map. |
| `agent.py` | PEAS: ClassroomEnvironment dan CleaningAgent dengan penyusunan ulang rencana (replanning). |

| Modul | Tanggung jawab |
|---|---|
| `api.py` | Jembatan JSON antara inti AI dan aplikasi 3D (spesifikasi denah → rencana). |
| `benchmark.py` | Benchmark skenario tetap (median 5 kali) dan uji skalabilitas. |
| `plots.py` | Grafik laporan dan presentasi dengan Matplotlib. |
| `demo.py` | Demo agen PEAS di terminal (peta ASCII, perencanaan ulang). |
| `export_static.py` | Ekspor data cadangan (denah, rencana, benchmark) ke folder web/data. |

Cuplikan pada Kode 4.1 memperlihatkan inti formulasi masalah: fungsi suksesor membangkitkan aksi bersihkan hanya bila posisi saat ini adalah target yang masih kotor, serta aksi gerak ke sel tetangga yang valid. Tabel tetangga valid dihitung sekali pada konstruktor sehingga pembangkitan suksesor tidak perlu memeriksa batas ruangan dan rintangan berulang kali.

**Kode 4.1** Fungsi suksesor dan uji tujuan pada CleaningProblem (disederhanakan).

```
def successors(self, s):                        # s = State(x, y, mask)
    out, bit = [], self._bit.get((s.x, s.y), 0)
    if s.mask & bit:                            # ada target kotor di sini
        out.append(("C", State(s.x, s.y, s.mask & ~bit),
                    self.clean_cost))
    for a, (nx, ny) in self._nbrs[(s.x, s.y)]:  # tetangga valid
        out.append((a, State(nx, ny, s.mask), self.move_cost))
    return out

def goal_test(self, s):
    return s.mask == 0 and (s.x, s.y) == self.home
```

Heuristik MST pada Kode 4.2 memakai medan jarak BFS yang dihitung satu kali untuk setiap target dan titik awal. Bobot MST dihitung dengan algoritma Prim atas himpunan target tersisa ditambah titik awal, lalu disimpan pada cache yang dikunci oleh mask sehingga himpunan target yang sama tidak dihitung ulang.

**Kode 4.2** Heuristik MST + jarak terdekat (disederhanakan).

```
def h_mst(s):
    p = s.y * W + s.x                          # indeks sel robot
    near = field_home[p]                       # jarak ke titik awal
    for i in bits(s.mask):                     # target yang tersisa
        near = min(near, fields_t[i][p])       # jarak ke simpul terdekat
    return mc * (mst_weight(s.mask) + near) + cc * popcount(s.mask)

def mst_weight(mask):                          # Prim atas R U {home}
    if mask in cache: return cache[mask]
    nodes = [HOME] + bits(mask)
    ...                                        # Prim dengan jarak BFS
    cache[mask] = total
    return total
```

Pencarian terbaik-pertama (UCS, A*, dan Greedy) diimplementasikan oleh satu fungsi generik dengan antrian prioritas (`heapq`). Kunci prioritas adalah g untuk UCS, pasangan (f, h) untuk A* sehingga seri diputus oleh h yang lebih kecil, dan h untuk Greedy. Penghapusan malas (*lazy deletion*) dipakai: simpul usang yang biayanya sudah tidak terbaik dilewati saat diambil dari antrian. Perekam ekspansi mencatat sel dari setiap state yang diperluas untuk membangun heat-map dan animasi pencarian. API aplikasi membatasi ukuran grid maksimum 40 × 40, jumlah target maksimum 12, dan jumlah ekspansi maksimum 250.000 agar permintaan yang tidak wajar tidak membebani server.

### 4.3 Implementasi Aset 3D dengan Blender

Seluruh aset dibuat dengan skrip Python yang dijalankan oleh Blender secara headless, tanpa pemodelan manual, sehingga dapat dibangun ulang dengan satu perintah. Skrip `make_textures.py` membangkitkan lima tekstur prosedural dengan NumPy (albedo, roughness, dan normal map lantai ubin; normal map plester dinding; serat kayu oak). Modul `lib.py` menyediakan kelas Asset yang membungkus bmesh dengan primitif kotak ber-bevel, silinder, bola, ikosfer, prisma bulat, bidang putar (*lathe*), bidang, mesh kustom, dan teks, ditambah fungsi pembuat material Principled BSDF dan ekspor glTF. Skrip `asset_robot.py`, `asset_props.py`, dan `asset_litter.py` menyusun robot, properti kelas, dan sampah; `build_assets.py` menjalankan semuanya, mengekspor tiga berkas .glb, dan merender gambar pratinjau.

**Tabel 4.3** Statistik aset 3D hasil ekspor glTF.

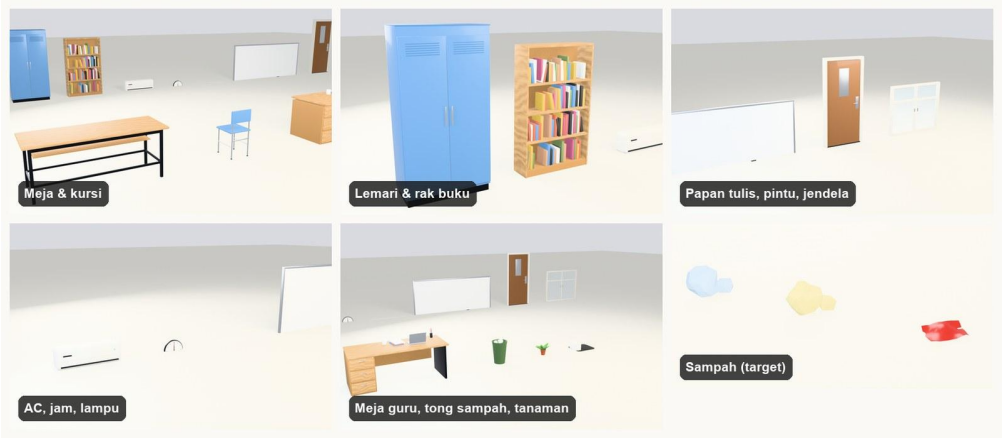
| Berkas | Objek mesh | Material PBR | Tekstur | Segitiga | Ukuran |
|---|---|---|---|---|---|
| `robot.glb` | 25 | 14 | 0 | 16.578 | 645 KB |
| `props.glb` | 71 | 43 | 1 | 27.752 | 1,19 MB |
| `litter.glb` | 17 | 17 | 0 | 1.810 | 124 KB |
| `Total` | 113 | 74 | 1 | 46.140 | 1,96 MB |

Robot CleanBot berbentuk kotak berpelat oranye dengan atap hitam, layar wajah, menara LiDAR di atas, dua roda bertapak karet, dua sikat samping, dan sikat utama. Bodi memakai cat oranye ber-*clearcoat*, ada strip LED cyan di

sekeliling bodi, dan layar wajah bermaterial emisif. Properti kelas meliputi meja bertekstur kayu, kursi, lemari berpintu ganda, rak buku berisi sekitar 80 buku acak, meja guru, papan tulis, pintu, jendela, jam dinding dengan jarum yang dapat digerakkan, AC, lampu, tempat sampah, tanaman, dan dok pengisian. Hasil pratinjau diperlihatkan pada Gambar 4.1 dan Gambar 4.2.



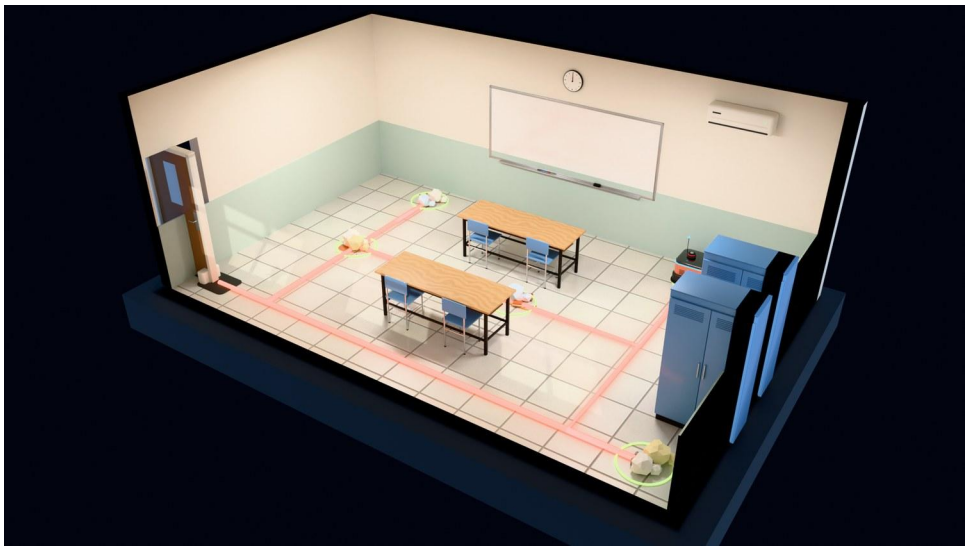

**Gambar 4.1** Model robot CleanBot hasil skrip Blender (depan, belakang, samping, atas).



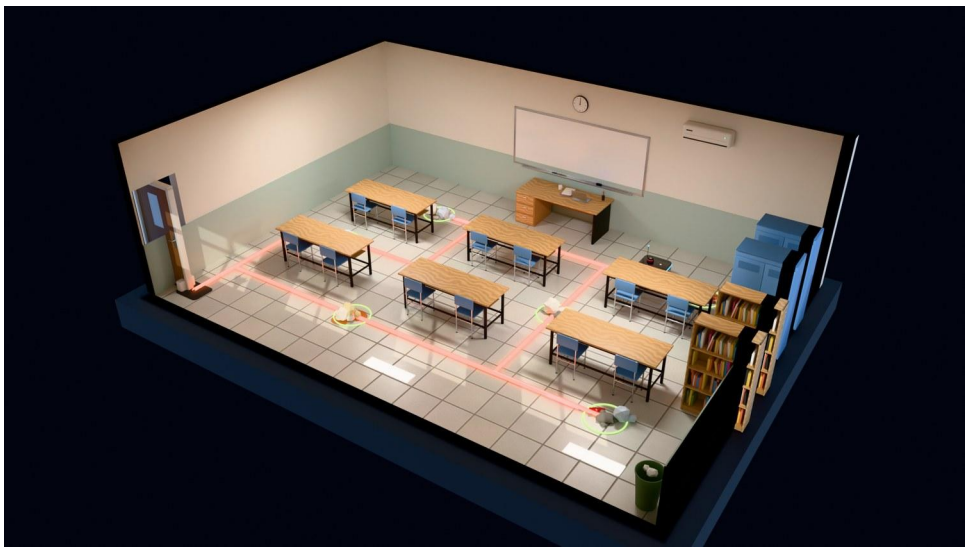

**Gambar 4.2** Properti ruang kelas dan model sampah hasil skrip Blender.

Skrip tambahan `render_scene.py` mewujudkan alur Python → Blender: skrip ini membaca denah (`web/data/meta.json`) dan rencana A* yang diekspor Python (`web/data/plans/S3_astar_mst.json`), menyusun ruang kelas dari aset .glb, menggambar rute rencana sebagai pita bercahaya di lantai, memasang lingkaran penanda pada titik kotoran, menempatkan robot pada posisi tertentu di

rute, lalu merender dengan Cycles (OptiX, 160 sampel dengan denoising). Agar ruang tampak seperti potongan diorama, dinding dan plafon sisi dekat kamera dibuat tidak terlihat oleh kamera namun tetap memblokir cahaya (properti `visible_camera`), sehingga pencahayaan matahari yang masuk lewat jendela tetap tampak realistis. Hasil render ditampilkan pada Gambar 4.3 dan Gambar 4.4, sedangkan detail robot pada render Cycles ditunjukkan oleh Gambar 4.5.



**Gambar 4.3** Render Cycles skenario S3 dengan rute rencana A* dan titik kotoran.



**Gambar 4.4** Render Cycles skenario S4 dengan robot di tengah rute.




**Gambar 4.5** Detail robot CleanBot pada render Cycles (kiri: S3, kanan: S4).

## 4.4 Implementasi Simulasi three.js

### *4.4.1 Rendering Realistis*

Aplikasi web dibangun dengan modul ES three.js r187 yang dimuat secara lokal. Penampil memakai `WebGLRenderer` dengan pemetaan nada `NeutralToneMapping` agar warna oranye robot tetap akurat. Pencahayaan terdiri atas cahaya matahari terarah dengan peta bayangan (sehingga bayangan jendela jatuh di lantai), cahaya lingkungan berbasis gambar dari `RoomEnvironment` yang diproses menjadi peta pantulan, cahaya hemisfer, dan lampu langit-langit. Hasil render melewati rantai pascapemrosesan `EffectComposer`: RenderPass, GTAOPass (ambient occlusion), UnrealBloomPass (cahaya berpendar pada LED dan jendela), dan OutputPass. Pengguna dapat memilih empat preset kualitas (Tabel 4.4) sesuai kemampuan perangkat.

**Tabel 4.4** Preset kualitas render pada aplikasi.

| Preset | Pixel ratio | Peta bayangan | AO | Bloom | MSAA |
|---|---|---|---|---|---|
| **Rendah** | 1 | 1024 px | tidak | tidak | – |
| **Sedang** | 1,25 | 2048 px | tidak | ya | 2× |
| **Tinggi** | 1,75 | 2048 px | ya | ya | 4× |
| **Ultra** | 2 | 4096 px | ya | ya | 4× |

Pada preset ultra dengan resolusi 1920 × 1080, pengujian pada GPU RTX 4060 Laptop (peramban Chrome tanpa kepala, ANGLE/Direct3D 11) menghasilkan sekitar 90 bingkai per detik, sehingga animasi tetap halus untuk keperluan presentasi. Angka ini diperoleh dari satu sesi pengujian dan dapat berbeda pada perangkat lain.

### *4.4.2 Ruang, Robot, dan Efek*

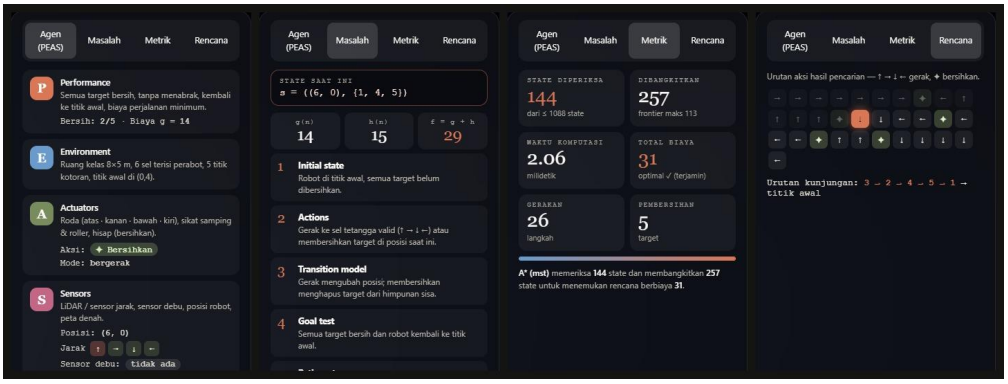
Ruang kelas dibangun oleh modul `room.js`: lantai ubin dengan peta normal, dinding berlubang untuk jendela dan pintu, plafon dengan lampu persegi, perabot yang ditempatkan dari denah S1–S4, serta dekorasi seperti papan tulis berisi teks prosedural (rumus A*, denah, dan catatan kelompok) dan poster PEAS. Dinding sisi dekat kamera memudar secara halus (*fade*) agar isi ruangan terlihat dari sudut mana pun. Robot memuat model Blender dan menganimasikan roda sesuai jarak tempuh, putaran sikat, putaran menara LiDAR, kemiringan bodi saat berakselerasi dan berbelok, warna LED menurut keadaan, serta layar wajah 512 × 256 piksel yang berganti ekspresi (diam, berpikir, bergerak, membersihkan, senang, siaga). Sampah tersedot ke dalam robot melalui animasi partikel. Efek suara sintetis (dengung motor, bip, dentang, hisap) dibangkitkan dengan WebAudio tanpa berkas audio.

### *4.4.3 Visualisasi Proses Pencarian*

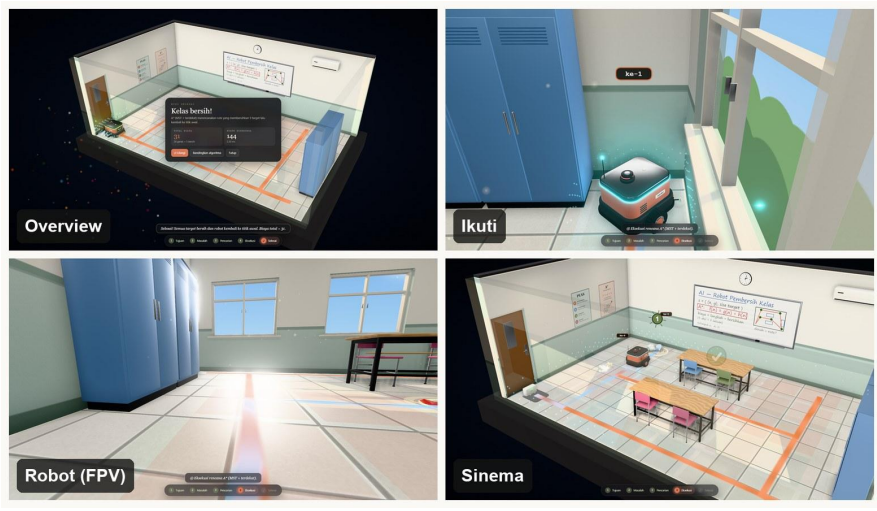
Modul `searchviz.js` menampilkan proses pencarian secara visual. Respons `/api/plan` memuat jumlah ekspansi tiap sel; sel-sel lantai digambar sebagai petak berwarna (instansiasi) dengan skala warna biru ke merah muda menurut banyaknya ekspansi, dan petak dimunculkan satu per satu menurut urutan ekspansi sehingga audiens melihat “gelombang” pencarian menyebar di lantai. Setelah pencarian selesai, rute rencana digambar sebagai pita bercahaya (*shader*) dan setiap target diberi penanda urutan kunjungan. Pada panel kanan, nilai g(n), h(n), dan f(n) diperbarui setiap langkah eksekusi; pada rencana optimal dengan heuristik consistent, f(n) tidak pernah menurun dan berakhir sama dengan biaya optimal. Gambar 4.6 memperlihatkan penanda tahap agen di bagian bawah layar, heat-map dan rute di lantai, panel PEAS dan state di kanan, serta panel kontrol di kiri. Keempat tab panel kanan ditunjukkan pada Gambar 4.7: Agen (PEAS), Masalah (state s beserta g, h, dan f), Metrik (hasil pencarian), dan Rencana (urutan aksi). Pengguna dapat berpindah di antara lima mode kamera (Gambar 4.8).




**Gambar 4.6** Tampilan aplikasi saat robot mengeksekusi rencana A* pada S3.




**Gambar 4.7** Empat tab panel kanan: PEAS, Masalah, Metrik, dan Rencana.



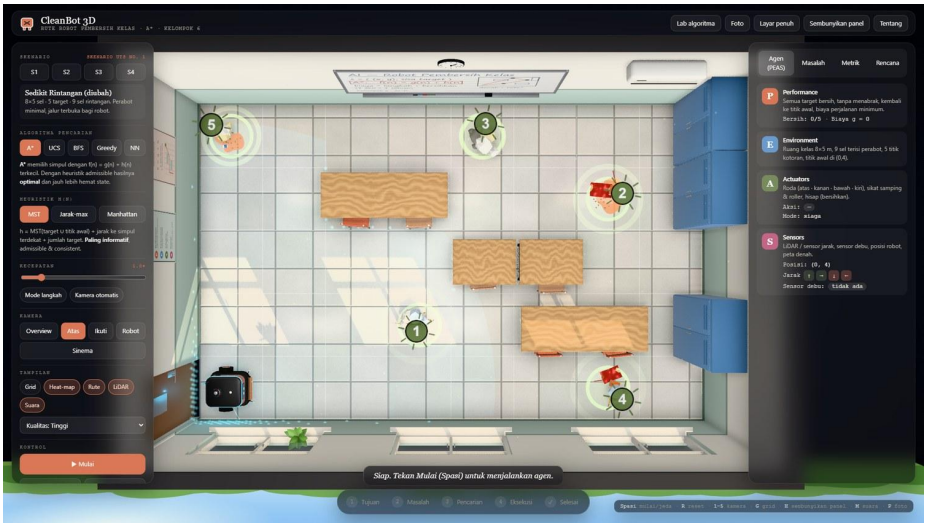

**Gambar 4.8** Empat dari lima mode kamera: overview, ikuti, FPV, dan sinema.

### *4.4.4 Mode Interaktif*

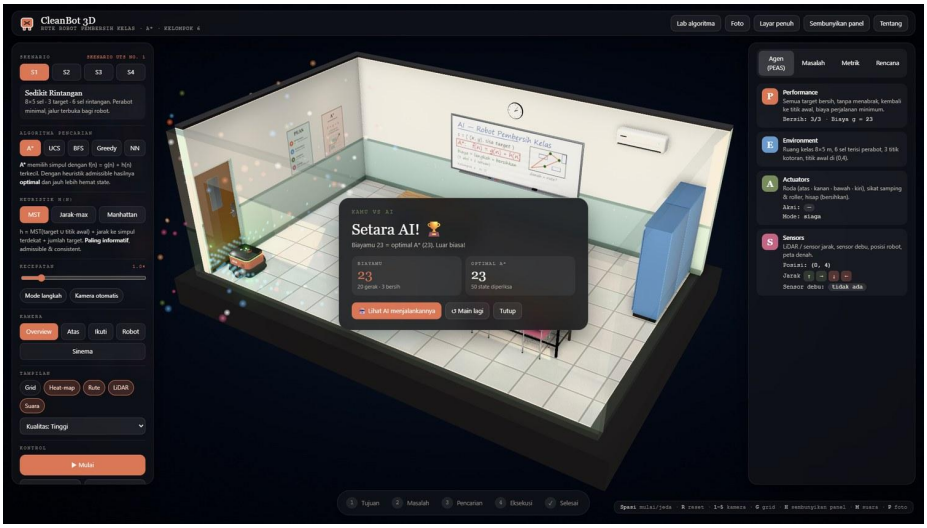
Selain demonstrasi otomatis, aplikasi menyediakan tiga fitur interaktif. Pertama, jendela Lab (Gambar 4.9) membandingkan seluruh algoritma pada S1–S4 dengan data dari `/api/benchmark` dan menampilkan grafik skalabilitas. Kedua, mode Editor (Gambar 4.10) memungkinkan pengguna menambah atau menghapus kotoran dan perabot dengan klik lantai atau mengacak denah, lalu menjalankan A* pada denah baru; target yang terhalang perabot dilaporkan sebagai target yang dilewati. Ketiga, mode Game "Kamu vs AI" (Gambar 4.11) membiarkan pengguna mengendalikan robot dengan tombol panah atau WASD dan membersihkan dengan spasi; biaya pengguna dibandingkan dengan biaya optimal A*, dan memainkan rencana optimal menghasilkan kartu "Setara AI".




**Gambar 4.9** Jendela Lab: perbandingan algoritma pada S1–S4.




**Gambar 4.10** Mode Editor: menambah kotoran dan perabot dengan klik lantai.




**Gambar 4.11** Mode Game "Kamu vs AI": hasil dibandingkan dengan rencana optimal A*.

## 4.5 Pengujian Fungsional

Kebenaran inti AI diverifikasi dengan 20 kasus uji otomatis (pytest) yang dirangkum pada Tabel 4.5. Eksekusi `python -m pytest tests -q` menghasilkan "20 passed" dalam sekitar 0,3 detik. Uji admissibility dan consistency mengambil 450 sampel state acak (6 denah acak × 3 heuristik × 25 state); untuk setiap sampel, biaya sisa optimal dihitung dengan UCS dari state tersebut dan dibandingkan dengan nilai h(s), serta ketidaksamaan $h(s) \leq c + h(s')$ diperiksa untuk seluruh suksesor.

**Tabel 4.5** Ringkasan pengujian otomatis.

| Pengujian | Hal yang diverifikasi | Kasus | Hasil |
|---|---|---|---|
| Algoritma optimal sepakat | BFS, UCS, dan A* (empat heuristik) berbiaya sama pada S1–S4; rencana valid saat diputar ulang dan mencapai tujuan. | 4 | Lulus |
| A* lebih hemat state | A* (MST) memeriksa tidak lebih dari UCS dan lebih sedikit dari BFS dengan biaya sama. | 4 | Lulus |
| Heuristik admissible dan consistent | $h(s) \leq$ biaya sisa optimal dan $h(s) \leq c + h(s')$ pada 450 sampel. | 1 | Lulus |
| Heuristik state tujuan | $h = 0$ pada state tujuan untuk semua heuristik. | 1 | Lulus |
| Tanpa target | Rencana kosong dan biaya nol untuk kelima algoritma. | 1 | Lulus |
| Titik awal sama dengan target | Rencana hanya satu aksi bersihkan berbiaya 1. | 1 | Lulus |
| Target tak terjangkau | Target terhalang dilewati dan dilaporkan; pencarian mentah gagal dengan pesan jelas. | 1 | Lulus |

| Pengujian | Hal yang diverifikasi | Kasus | Hasil |
|---|---|---|---|
| Nearest-Neighbor tidak mengalahkan optimal | Pada 15 denah acak, biaya NN selalu ≥ optimal dan ada denah dengan biaya lebih tinggi. | 1 | Lulus |
| Agen menyelesaikan tugas | Tugas selesai, nol tabrakan, biaya sama dengan optimal, satu kali perencanaan pada S1–S4. | 4 | Lulus |
| Penyusunan ulang rencana | Rintangan tak terduga memicu perencanaan ulang (≥ 2) tanpa tabrakan. | 1 | Lulus |
| Katalog sesuai UTS | S1–S3 berukuran 8×5 dengan 3, 3, dan 5 target serta titik awal (0, 4). | 1 | Lulus |

## 4.6 Hasil Eksperimen

### *4.6.1 Hasil pada Skenario S1–S4*

Tabel 4.6 menunjukkan bahwa BFS, UCS, dan ketiga varian A* menghasilkan biaya yang sama pada setiap skenario, yaitu 23 (S1), 21 (S2), 31 (S3), 38 (S4) aksi, yang berarti semuanya optimal. Greedy dan Nearest-Neighbor juga optimal pada S1–S3, tetapi pada S4 menghasilkan biaya 42, yaitu 10,5% lebih mahal. Dalam seluruh kasus, tugas selesai dan robot kembali ke titik awal.

**Tabel 4.6** Total biaya rencana (jumlah aksi) pada tiap skenario.

| Algoritma | S1 | S2 | S3 | S4 |
|---|---|---|---|---|
| **BFS** | 23 | 21 | 31 | 38 |
| **UCS** | 23 | 21 | 31 | 38 |
| **A* (Manhattan)** | 23 | 21 | 31 | 38 |
| **A* (jarak-max)** | 23 | 21 | 31 | 38 |
| **A* (MST)** | 23 | 21 | 31 | 38 |
| **Greedy (MST)** | 23 | 21 | 31 | 42 |
| **Nearest-Neighbor** | 23 | 21 | 31 | 42 |

*Biaya optimal: S1 = 23, S2 = 21, S3 = 31, S4 = 38. Nilai yang lebih tinggi dari optimal hanya terdapat pada Greedy dan Nearest-Neighbor di S4.*

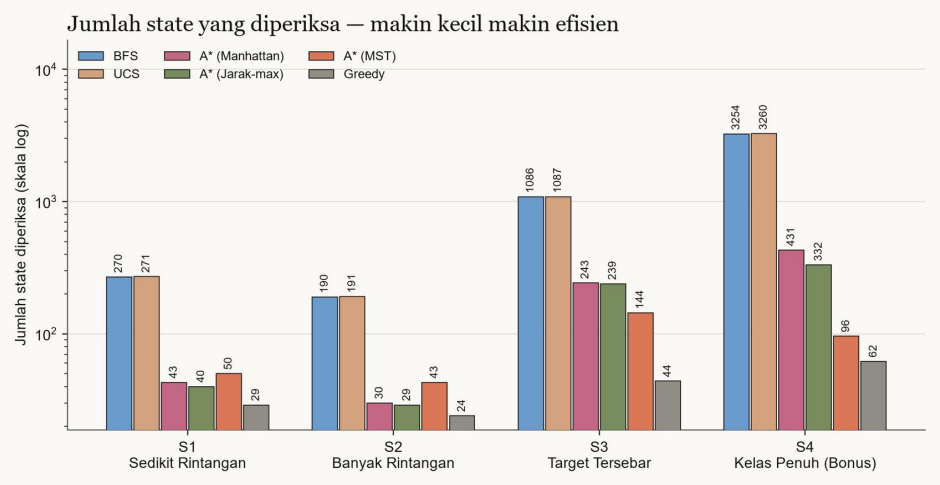
Jumlah state yang diperiksa ditunjukkan pada Tabel 4.7 dan Gambar 4.12. Jumlah ini jauh berbeda antar-algoritma meskipun biayanya sama. BFS dan UCS memeriksa hampir seluruh ruang state yang dapat dicapai pada kedalaman sebelum tujuan (270 state pada S1, 190 state pada S2, 1.086 state pada S3, 3.254 state pada S4), sedangkan A* dengan heuristik MST hanya memeriksa 50, 43, 144, 96 state. UCS memeriksa sedikit lebih banyak state daripada BFS (misalnya 3260

dibanding 3.254 pada S4) karena uji tujuannya dilakukan saat simpul diperluas, bukan saat dibangkitkan.

**Tabel 4.7** Jumlah state yang diperiksa (diperluas) pada tiap skenario.

| Algoritma | S1 | S2 | S3 | S4 |
|---|---|---|---|---|
| **BFS** | 270 | 190 | 1.086 | 3.254 |
| **UCS** | 271 | 191 | 1.087 | 3.260 |
| **A* (Manhattan)** | 43 | 30 | 243 | 431 |
| **A* (jarak-max)** | 40 | 29 | 239 | 332 |
| **A* (MST)** | 50 | 43 | 144 | 96 |
| **Greedy (MST)** | 29 | 24 | 44 | 62 |
| **Nearest-Neighbor** | – | – | – | – |

*Nearest-Neighbor bukan pencarian pada ruang state sehingga tidak memiliki hitungan state diperiksa.*



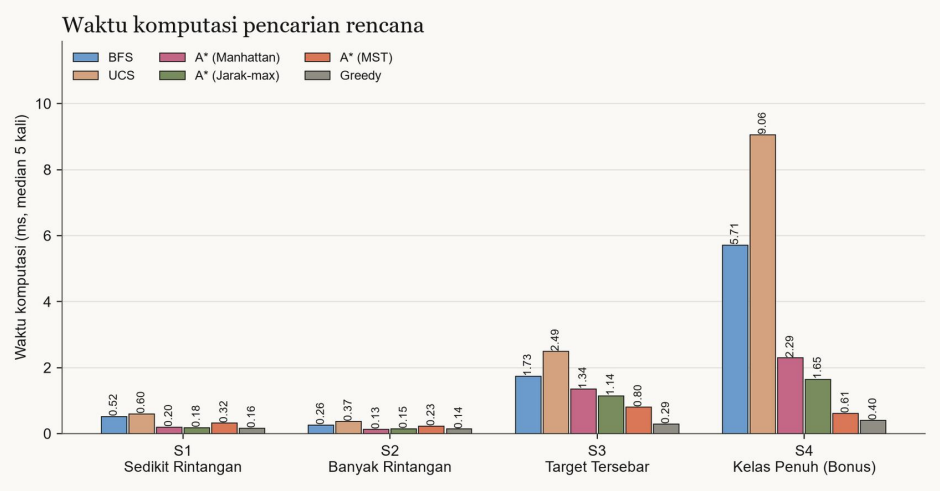

**Gambar 4.12** Jumlah state yang diperiksa tiap algoritma (skala logaritmik).

Rasio penghematan A* (MST) terhadap BFS adalah 5,4× (S1), 4,4× (S2), 7,5× (S3), 33,9× (S4), dan terhadap UCS 5,4×, 4,4×, 7,5×, 34,0×. Penghematan terbesar terjadi pada S4 yang memiliki ruang state terluas (3.264 state) dan jumlah target terbanyak.

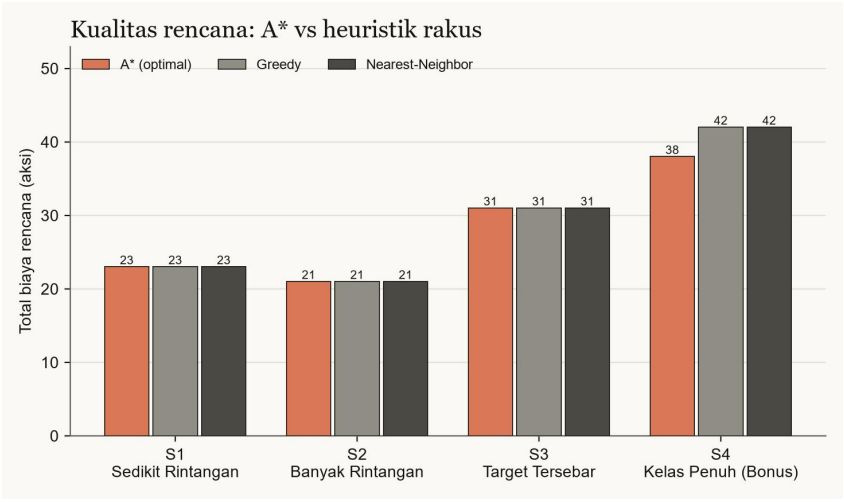
Waktu komputasi diperlihatkan pada Tabel 4.8 dan Gambar 4.13. Seluruh algoritma menyelesaikan pencarian di bawah 10 milidetik, sehingga layak untuk penyusunan ulang rencana secara real-time. A* (MST) lebih cepat daripada BFS dan UCS pada keempat skenario: pada S4 hanya 0,61 ms dibanding 5,71 ms (BFS) dan 9,06 ms (UCS). Greedy dan Nearest-Neighbor adalah yang tercepat, tetapi tidak menjamin biaya optimal.

**Tabel 4.8** Waktu komputasi pencarian (ms, median 5 kali jalan).

| Algoritma | S1 | S2 | S3 | S4 |
|---|---|---|---|---|
| **BFS** | 0,52 | 0,26 | 1,73 | 5,71 |
| **UCS** | 0,60 | 0,37 | 2,49 | 9,06 |
| **A* (Manhattan)** | 0,20 | 0,13 | 1,34 | 2,29 |
| **A* (jarak-max)** | 0,18 | 0,15 | 1,14 | 1,65 |
| **A* (MST)** | 0,32 | 0,23 | 0,80 | 0,61 |
| **Greedy (MST)** | 0,16 | 0,14 | 0,29 | 0,40 |
| **Nearest-Neighbor** | 0,04 | 0,03 | 0,09 | 0,07 |




**Gambar 4.13** Waktu komputasi pencarian tiap algoritma.




**Gambar 4.14** Total biaya rencana: A* dibanding Greedy dan Nearest-Neighbor.

### *4.6.2 Pengaruh Heuristik*

Tabel 4.9 membandingkan nilai ketiga heuristik pada state awal dengan biaya optimal C*, serta jumlah state yang diperiksa A* untuk masing-masing

heuristik. Ketiganya tidak pernah melebihi C*, sesuai sifat admissible. Pada S1 dan S2 yang hanya memiliki tiga target, heuristik jarak-max memberi nilai awal yang lebih dekat ke C* (21 dan 17) daripada MST (14 dan 12), sehingga A* dengan jarak-max memeriksa sedikit lebih sedikit state (40 dan 29 dibanding 50 dan 43).

**Tabel 4.9** Nilai heuristik pada state awal dan jumlah state yang diperiksa A*.

| ID | C* | $h_M(s_0)$ | Diperiksa (Manhattan) | $h_{max}(s_0)$ | Diperiksa (jarak-max) | $h_{MST}(s_0)$ | Diperiksa (MST) |
|---|---|---|---|---|---|---|---|
| **S1** | 23 | 21 | 43 | 21 | 40 | 14 | 50 |
| **S2** | 21 | 17 | 30 | 17 | 29 | 12 | 43 |
| **S3** | 31 | 25 | 243 | 25 | 239 | 22 | 144 |
| **S4** | 38 | 32 | 431 | 32 | 332 | 32 | 96 |

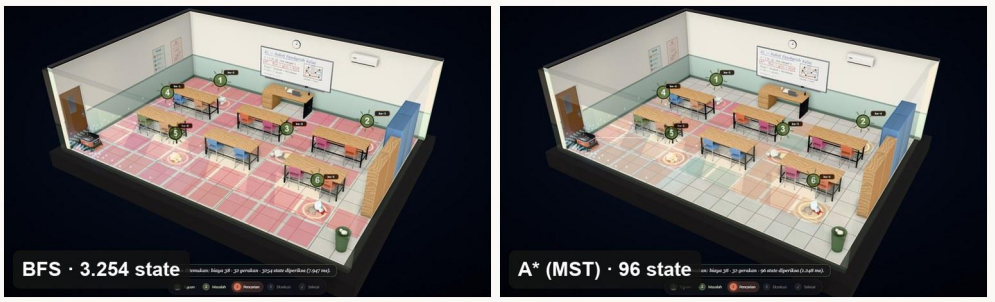
Sebaliknya, pada S3 dan S4 yang memiliki lima dan enam target, A* dengan MST memeriksa jauh lebih sedikit state (144 dan 96) daripada jarak-max (239 dan 332) dan Manhattan (243 dan 431), meskipun nilai h pada state awal tidak lebih tinggi (pada S4 ketiganya bernilai 32). Penjelasannya, jarak-max dan Manhattan hanya memperhitungkan target yang paling jauh, sehingga nilainya turun tajam setelah target terjauh dibersihkan dan banyak jalur yang tidak menjanjikan masih tampak murah. Heuristik MST memperhitungkan keterhubungan seluruh target yang tersisa sehingga nilainya tetap informatif pada state-state antara. Dengan kata lain, kualitas heuristik perlu dinilai pada seluruh state yang dijelajahi, bukan hanya pada state awal.

### *4.6.3 Kualitas Rencana Heuristik Rakus*

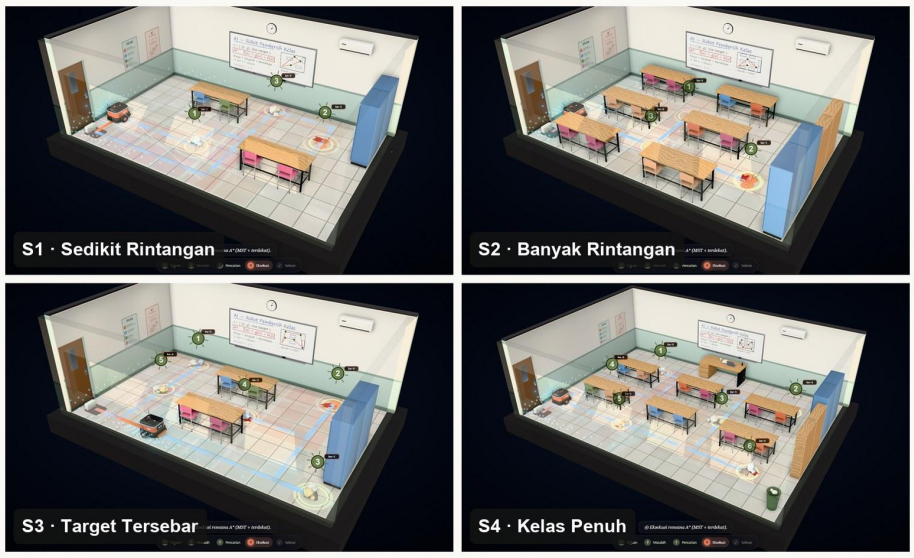
Nearest-Neighbor dan Greedy menghasilkan rencana yang valid dengan komputasi tercepat, tetapi tidak selalu optimal. Pada S4, urutan kunjungan optimal A* adalah 5 → 6 → 3 → 2 → 1 → 4 dengan biaya 38, sedangkan Nearest-Neighbor menempuh urutan 4 → 5 → 1 → 2 → 3 → 6 dengan biaya 42. Pada S2 dan S3, urutan kunjungannya berbeda dari urutan optimal tetapi biayanya sama (21 dan 31), sehingga selisih hanya tampak ketika denah cukup rumit. Temuan ini menegaskan bahwa urutan kunjungan menentukan biaya total dan bahwa menuju target terdekat tidak menjamin tur terpendek.

### *4.6.4 Peta Panas Ekspansi State*

Gambar 4.15 membandingkan heat-map ekspansi BFS dan A* pada S4. Pada BFS hampir seluruh lantai berwarna karena pencarian meluas ke segala arah tanpa petunjuk; pada A* hanya sel-sel di sekitar rute yang diperiksa karena h(n) mengarahkan pencarian ke target. Gambar 4.16 memperlihatkan rute optimal A* pada keempat skenario dalam tampilan 3D.




**Gambar 4.15** Heat-map ekspansi state pada S4: BFS (kiri) dan A* MST (kanan).




**Gambar 4.16** Rute optimal A* (pita oranye) pada S1–S4 dalam tampilan 3D.

### *4.6.5 Uji Skalabilitas*

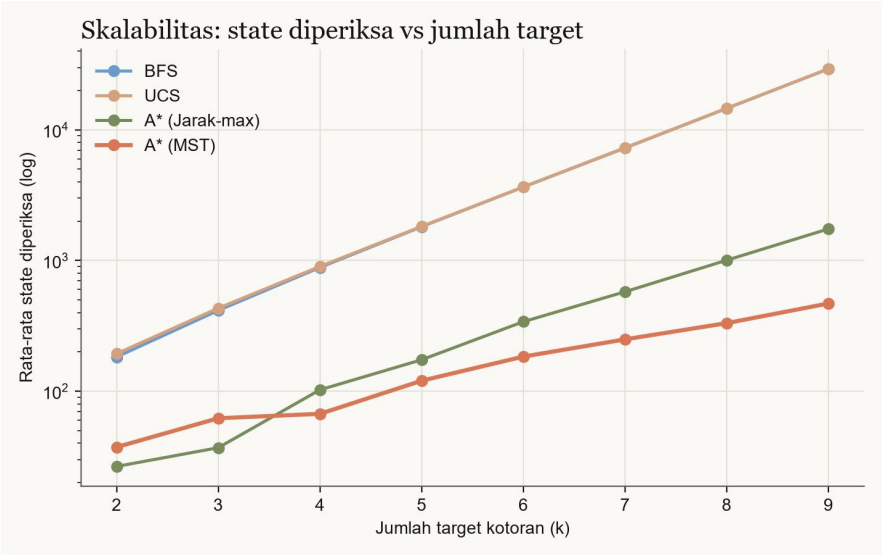
Tabel 4.10 dan Gambar 4.17 menunjukkan hasil uji skalabilitas pada denah acak 10 × 7 dengan 2 sampai 9 target. Jumlah state yang diperiksa BFS dan UCS meningkat sekitar dua kali lipat untuk setiap tambahan satu target (faktor pertumbuhan rata-rata 2,01× per target pada k = 4 sampai 9), sesuai batas atas $2^k$. A* dengan heuristik MST tumbuh jauh lebih lambat (1,48× per target), sedangkan A* dengan jarak-max berada di antaranya (1,76×). Pada k = 9, A* (MST)

memeriksa rata-rata 470 state dibanding 29.256 state pada BFS, yaitu 62 kali lebih sedikit, dengan waktu 3,4 ms dibanding 60,6 ms (17,8 kali lebih cepat).

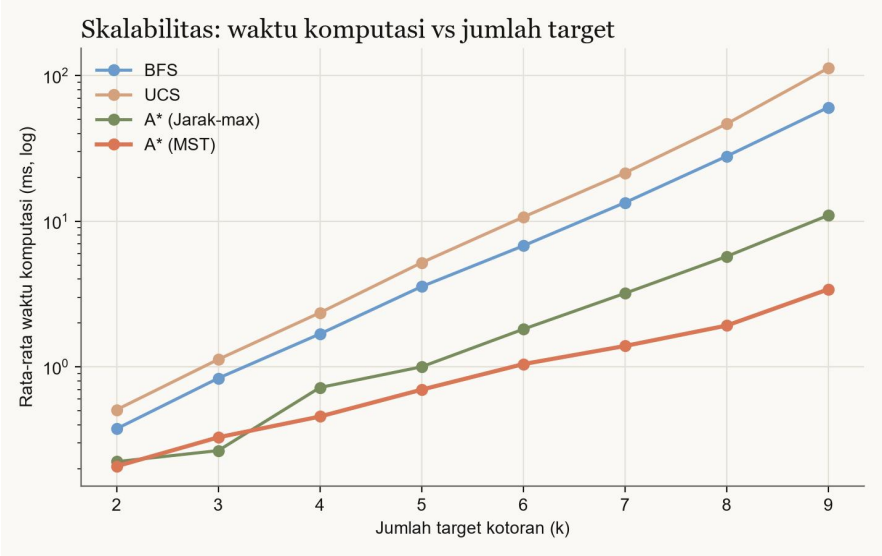
**Tabel 4.10** Hasil uji skalabilitas pada denah acak 10 × 7.

| k | BFS | UCS | A* (jarak-max) | A* (MST) | BFS ÷ A* (MST) | Waktu BFS (ms) | Waktu A* MST (ms) |
|---|---|---|---|---|---|---|---|
| 2 | 183 | 194 | 27 | 37 | 4,9× | 0,4 | 0,2 |
| 3 | 415 | 432 | 37 | 62 | 6,7× | 0,8 | 0,3 |
| 4 | 886 | 902 | 103 | 67 | 13,2× | 1,7 | 0,5 |
| 5 | 1.815 | 1.827 | 174 | 120 | 15,1× | 3,6 | 0,7 |
| 6 | 3.652 | 3.659 | 341 | 184 | 19,8× | 6,8 | 1,0 |
| 7 | 7.264 | 7.297 | 577 | 250 | 29,1× | 13,4 | 1,4 |
| 8 | 14.592 | 14.625 | 1.007 | 332 | 43,9× | 28,0 | 1,9 |
| 9 | 29.256 | 29.279 | 1.746 | 470 | 62,3× | 60,6 | 3,4 |

*Rata-rata dari 5 denah acak (benih 0–4) untuk setiap k; BFS ÷ A* (MST) adalah rasio jumlah state yang diperiksa.*




**Gambar 4.17** Rata-rata state diperiksa terhadap jumlah target.




**Gambar 4.18** Rata-rata waktu komputasi terhadap jumlah target.

### *4.6.6 Penyusunan Ulang Rencana (Replanning)*

Untuk menguji kemampuan agen menangani hal tak terduga, satu sel pada rute optimal dibuat tertutup rintangan baru tepat setelah langkah ke-2 (Tabel 4.11). Rintangan tersebut tidak diketahui agen pada perencanaan awal. Sensor jarak mendeteksinya ketika robot berada di samping sel itu, rencana lama dibuang, dan agen merumuskan ulang masalah dari posisinya sekarang dengan pengetahuan yang telah diperbarui. Pada ketiga skenario yang diuji, tugas tetap selesai, perencanaan dilakukan dua kali, dan tidak terjadi tabrakan. Biaya akhir sama dengan biaya awal pada S1 dan S3 (terdapat rute alternatif berbiaya sama), dan hanya naik 2 satuan pada S4 karena rute yang tertutup memaksa jalan memutar.

**Tabel 4.11** Hasil uji penyusunan ulang rencana dengan rintangan tak terduga.

| Skenario | Sel yang tertutup | Biaya awal | Biaya akhir | Jumlah perencanaan | State diperiksa (per perencanaan) | Tabrakan |
|---|---|---|---|---|---|---|
| **S1** | (3, 2) | 23 | 23 | 2 | 50 + 39 | 0 |
| **S3** | (6, 3) | 31 | 31 | 2 | 144 + 28 | 0 |
| **S4** | (4, 5) | 38 | 40 | 2 | 96 + 114 | 0 |

## 4.7 Pembahasan

**Pemodelan masalah (rumusan masalah 1).** Tugas pembersihan multi-target dapat dirumuskan secara bersih dengan state (x, y, mask) sehingga lima komponen formulasi langsung dapat diterjemahkan ke kode. Representasi bitmask

membuat ruang state kecil dan terukur: paling banyak 3.264 state pada skenario terbesar (S4). Agen yang mengimplementasikan SIMPLE-PROBLEM-SOLVING-AGENT menyelesaikan semua skenario tanpa tabrakan dengan biaya yang sama dengan rencana optimal, dan mampu merencanakan ulang ketika sensor menemukan rintangan baru.

**Perbandingan algoritma (rumusan masalah 2).** Hasil sesuai teori: BFS, UCS, dan A* optimal sehingga biayanya identik. BFS dan UCS nyaris sama karena biaya langkah seragam, dan hanya berbeda tipis pada jumlah ekspansi akibat perbedaan saat uji tujuan. A* secara konsisten lebih efisien: 4,4 hingga 33,9 kali lebih sedikit state dan lebih cepat pada seluruh skenario. Karena memori pencarian berbanding lurus dengan jumlah state yang dicapai, A* juga cenderung lebih ringan dalam penggunaan memori.

**Pengaruh jumlah target dan heuristik (rumusan masalah 3).** Ruang state tumbuh eksponensial terhadap jumlah target, dan hal ini terlihat pada BFS/UCS yang menggandakan jumlah state setiap tambahan satu target. Heuristik yang informatif memperlambat pertumbuhan itu: A* dengan MST tumbuh sekitar 1,5× per target. Pemilihan heuristik bergantung pada kondisi: dengan sedikit target, estimasi jarak-max yang sederhana sudah cukup dan sedikit lebih baik, sedangkan dengan banyak target MST jauh lebih unggul. Heuristik rakus (Greedy dan Nearest-Neighbor) sangat cepat dan sering optimal pada denah sederhana, tetapi dapat meleset ketika urutan kunjungan menjadi penentu, seperti pada S4 (+10,5%).

**Visualisasi 3D (rumusan masalah 4).** Pipeline Python → Blender → three.js berhasil memisahkan tanggung jawab: Python menjamin kebenaran dan optimalitas rencana, Blender menyediakan aset berkualitas tinggi yang dapat dibangun ulang dengan skrip, dan three.js menyajikannya secara interaktif. Heat-map ekspansi, penanda tahap agen, serta panel g(n), h(n), f(n) mengubah angka benchmark menjadi pengalaman visual yang mudah dijelaskan di depan audiens. Waktu perencanaan di bawah 10 milidetik membuat penyusunan ulang rencana pada mode Editor terasa instan, sehingga waktu tunggu pengguna lebih ditentukan oleh animasi daripada oleh komputasi.

## 4.8 Keterbatasan

1. Lingkungan diasumsikan teramati penuh dan statis; hanya rintangan tak terduga yang diuji lewat penyusunan ulang rencana, belum objek bergerak seperti siswa.
2. Biaya 1 per aksi merupakan asumsi; energi sebenarnya bergantung pada belokan, kecepatan, dan jenis lantai.
3. Pertumbuhan ruang state bersifat eksponensial; aplikasi dibatasi 12 target. Untuk target yang jauh lebih banyak diperlukan heuristik yang lebih kuat atau pembagian zona.
4. Grid empat arah tidak memodelkan gerak diagonal atau lengkung robot nyata, dan simulasi 3D belum memuat dinamika fisik.
5. Kualitas visual dinilai secara kualitatif; belum ada uji pengguna untuk mengukur efektivitas visualisasi dalam pembelajaran.
6. Kecepatan bingkai hanya diukur pada satu perangkat dan satu peramban.

# BAB V
# PENUTUP

## 5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan evaluasi yang telah dilakukan, diperoleh kesimpulan berikut.

1. Tugas pembersihan ruang kelas berhasil dimodelkan sebagai masalah *problem solving* melalui PEAS dan lima komponen formulasi dengan state s = (x, y, mask). Agen berbasis tujuan yang mengimplementasikan SIMPLE-PROBLEM-SOLVING-AGENT menyelesaikan seluruh skenario S1–S4 tanpa tabrakan dan mampu menyusun ulang rencana ketika menemukan rintangan baru.
2. BFS, UCS, dan A* menghasilkan rencana optimal berbiaya sama (23, 21, 31, 38 aksi pada S1–S4). A* dengan heuristik MST memeriksa 4,4 hingga 33,9 kali lebih sedikit state daripada BFS dan lebih cepat pada semua skenario (0,61 ms pada S4).
3. Ketiga heuristik admissible dan consistent (diverifikasi pada 450 sampel state). Pada tiga target heuristik jarak-max sudah memadai, sedangkan pada lima sampai enam target heuristik MST jauh lebih efisien. Pada uji skalabilitas, BFS dan UCS tumbuh sekitar dua kali lipat per target, sedangkan A* (MST) sekitar 1,5 kali; pada sembilan target A* (MST) memeriksa 62 kali lebih sedikit state.
4. Urutan kunjungan menentukan biaya total: Greedy dan Nearest-Neighbor menghasilkan biaya 42 pada S4, yaitu 10,5% lebih mahal dari optimal (38).
5. Pipeline Python → Blender → three.js menghasilkan simulasi 3D yang realistis dan interaktif dengan aset ±46 ribu segitiga, material PBR, bayangan matahari, ambient occlusion, serta visualisasi heat-map pencarian, mode Editor, dan permainan “Kamu vs AI”, sehingga proses berpikir agen dapat dijelaskan secara visual. Seluruh 20 pengujian otomatis dinyatakan lulus.

## 5.2 Saran

1. **Baterai dan dok pengisian**: menambahkan kapasitas baterai sehingga dok pengisian menjadi tujuan antara bila energi menipis.
2. **Banyak robot**: membagi target ke beberapa robot dengan pengelompokan K-Means (*cluster-first, route-second*) lalu merencanakan rute tiap kelompok dengan A*.
3. **Rintangan dinamis**: memakai penyusunan ulang rencana inkremental seperti D* Lite untuk menangani siswa atau kursi yang berpindah.
4. **Cakupan lantai**: menerapkan *coverage path planning* agar seluruh permukaan lantai dibersihkan, bukan hanya titik kotoran tertentu.
5. **Skala lebih besar**: memakai heuristik yang lebih kuat, IDA*, atau pembagian zona bila jumlah target melebihi batas aplikasi.
6. **Robot nyata dan uji pengguna**: menguji pada robot sungguhan (ROS, LiDAR, dan kamera pendeteksi kotoran) serta melakukan uji pengguna untuk menilai efektivitas visualisasi dalam pembelajaran.

# DAFTAR PUSTAKA


Burley, B. (2012). Physically-based shading at Disney. *ACM SIGGRAPH 2012 Course: Practical Physically-Based Shading in Film and Game Production*. Walt Disney Animation Studios.

Dechter, R., & Pearl, J. (1985). Generalized best-first search strategies and the optimality of A*. *Journal of the ACM, 32*(3), 505–536.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107.

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics, 10*(1), 196–210.

Moon, W., Park, B., Nengroo, S. H., Kim, T., & Har, D. (2022). Path planning of cleaning robot with reinforcement learning. *arXiv preprint arXiv:2208.08211*.

National Institute of Standards and Technology. (n.d.). *Mobility performance of robotic systems*. NIST.

Prim, R. C. (1957). Shortest connection networks and some generalizations. *The Bell System Technical Journal, 36*(6), 1389–1401.

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

University of California, Berkeley. (n.d.). *CS188: Introduction to artificial intelligence — Uninformed search and informed search*. Course materials.

Blender Foundation. (n.d.). *Blender Python API documentation (bpy)*. https://docs.blender.org/api/current/

Khronos Group. (n.d.). *glTF 2.0 specification*. https://registry.khronos.org/glTF/

three.js authors. (n.d.). *three.js documentation*. https://threejs.org/docs/

Ramírez, S. (n.d.). *FastAPI documentation*. https://fastapi.tiangolo.com/

# LAMPIRAN A
# PETUNJUK MENJALANKAN APLIKASI

Aplikasi dijalankan pada Windows, macOS, atau Linux dengan Python 3.10 atau lebih baru dan peramban modern (disarankan Chrome atau Edge dengan akselerasi GPU). Aplikasi tidak memerlukan koneksi internet karena seluruh aset dan pustaka three.js berada di folder `web/`.

## A.1 Instalasi dan Menjalankan

```
cd UAS_CleanBot3D
pip install -r requirements.txt     # sekali saja
python server.py --open               # atau klik dua kali run.bat
```

Peramban terbuka pada alamat http://localhost:8765. Tekan spasi untuk memulai agen. Bila server Python tidak aktif, aplikasi memakai data cadangan sehingga skenario S1–S4 tetap dapat dijalankan.

## A.2 Kontrol Aplikasi

**Tabel A.1** Tombol kontrol aplikasi.

| Tombol | Fungsi | Tombol | Fungsi |
|---|---|---|---|
| **Spasi** | Mulai / jeda / lanjutkan | 1 – 5 | Kamera: overview, atas, ikuti, robot (FPV), sinema |
| **R** | Reset | G / L | Tampilkan grid / LiDAR |
| **S** | Satu langkah (mode langkah) | H | Sembunyikan panel (mode presentasi bersih) |
| + / – | Atur kecepatan | M / P | Suara / simpan gambar |
| **Panah atau WASD** | Gerak robot (mode Game) | Spasi (Game) | Bersihkan titik kotoran |

## A.3 Perintah Pengujian dan Pembangunan Ulang

```
python -m pytest tests -q                    # 20 pengujian otomatis
python -m cleanbot.benchmark                 # benchmark + grafik
(results/)
python -m cleanbot.demo S3 --algo astar      # agen PEAS di terminal
python blender/make_textures.py              # tekstur prosedural (NumPy)
blender --background --factory-startup --python blender/build_assets.py
blender --background --factory-startup --python blender/render_scene.py
-- --scenario S3
python -m cleanbot.export_static             # data cadangan web/data/
```

# LAMPIRAN B
# STRUKTUR PROYEK

```
UAS_CleanBot3D/
├── cleanbot/          inti AI (Python)
│   ├── scenarios.py       denah S1–S4 + generator denah acak
│   ├── problem.py         lima komponen formulasi masalah
│   ├── heuristics.py      Manhattan, jarak-max, MST + terdekat
│   ├── search.py          BFS, UCS, A*, Greedy, Nearest-Neighbor
│   ├── agent.py           PEAS + agen pemecah masalah + replanning
│   ├── api.py             jembatan JSON ke aplikasi 3D
│   ├── benchmark.py       benchmark + uji skalabilitas
│   └── plots.py           grafik laporan dan presentasi
├── server.py          FastAPI: /api/meta, /api/plan, /api/benchmark
├── web/               aplikasi three.js (index.html, js/, css/,
│                      assets/, vendor/, data/)
├── blender/           skrip bpy: build_assets, asset_*, render_scene
├── tests/             20 pengujian otomatis (pytest)
├── tools/             capture.py (tangkapan layar otomatis)
├── results/           CSV/JSON benchmark dan grafik
└── docs/              tangkapan layar dan render pratinjau
```

Jumlah kode: inti AI Python sekitar 1.900 baris, aplikasi web (JavaScript) sekitar 4.200 baris, skrip Blender sekitar 1.600 baris, dan pengujian sekitar 170 baris.

# LAMPIRAN C
# HASIL BENCHMARK LENGKAP

**Tabel C.1** Hasil benchmark seluruh konfigurasi pada skenario S1–S4.

| ID | Algoritma | Biaya | Gerak | Diperiksa | Dibang-kitkan | Frontier maks | Waktu (ms) | Selisih |
|---|---|---|---|---|---|---|---|---|
| S1 | BFS | 23 | 20 | 270 | 271 | 29 | 0,52 | 0% |
| S1 | UCS | 23 | 20 | 271 | 271 | 29 | 0,60 | 0% |
| S1 | A* (Manhattan) | 23 | 20 | 43 | 75 | 33 | 0,20 | 0% |
| S1 | A* (jarak-max) | 23 | 20 | 40 | 72 | 33 | 0,18 | 0% |
| S1 | A* (MST) | 23 | 20 | 50 | 80 | 30 | 0,32 | 0% |
| S1 | Greedy (MST) | 23 | 20 | 29 | 54 | 26 | 0,16 | 0% |
| S1 | Nearest-Neighbor | 23 | 20 | 0 | 0 | 0 | 0,04 | 0% |
| S2 | BFS | 21 | 18 | 190 | 191 | 21 | 0,26 | 0% |
| S2 | UCS | 21 | 18 | 191 | 191 | 21 | 0,37 | 0% |
| S2 | A* (Manhattan) | 21 | 18 | 30 | 48 | 19 | 0,13 | 0% |
| S2 | A* (jarak-max) | 21 | 18 | 29 | 48 | 20 | 0,15 | 0% |
| S2 | A* (MST) | 21 | 18 | 43 | 76 | 33 | 0,23 | 0% |
| S2 | Greedy (MST) | 21 | 18 | 24 | 47 | 24 | 0,14 | 0% |
| S2 | Nearest-Neighbor | 21 | 18 | 0 | 0 | 0 | 0,03 | 0% |
| S3 | BFS | 31 | 26 | 1.086 | 1.087 | 90 | 1,73 | 0% |
| S3 | UCS | 31 | 26 | 1.087 | 1.087 | 90 | 2,49 | 0% |
| S3 | A* (Manhattan) | 31 | 26 | 243 | 335 | 86 | 1,34 | 0% |
| S3 | A* (jarak-max) | 31 | 26 | 239 | 327 | 85 | 1,14 | 0% |
| S3 | A* (MST) | 31 | 26 | 144 | 257 | 113 | 0,80 | 0% |
| S3 | Greedy (MST) | 31 | 26 | 44 | 83 | 40 | 0,29 | 0% |
| S3 | Nearest-Neighbor | 31 | 26 | 0 | 0 | 0 | 0,09 | 0% |
| S4 | BFS | 38 | 32 | 3.254 | 3.260 | 218 | 5,71 | 0% |
| S4 | UCS | 38 | 32 | 3.260 | 3.262 | 218 | 9,06 | 0% |
| S4 | A* (Manhattan) | 38 | 32 | 431 | 654 | 209 | 2,29 | 0% |
| S4 | A* (jarak-max) | 38 | 32 | 332 | 556 | 212 | 1,65 | 0% |
| S4 | A* (MST) | 38 | 32 | 96 | 190 | 95 | 0,61 | 0% |
| S4 | Greedy (MST) | 42 | 36 | 62 | 122 | 61 | 0,40 | 10,5% |
| S4 | Nearest-Neighbor | 42 | 36 | 0 | 0 | 0 | 0,07 | 10,5% |

*Waktu adalah median 5 kali jalan. Selisih adalah persentase biaya di atas biaya optimal.*

# LAMPIRAN D
# CUPLIKAN KODE INTI

Berikut versi ringkas fungsi pencarian terbaik-pertama pada `search.py` yang melayani UCS, A*, dan Greedy.

**Kode D.1** Pencarian terbaik-pertama generik (disederhanakan dari search.py).

```
def best_first(problem, mode, h):          # mode: ucs | astar | greedy
    start = problem.initial_state()
    best_g, parent = {start: 0}, {start: None}
    frontier, closed = [], set()
    push(frontier, key(0, start), 0, start)
    while frontier:
        _, _, g, s = heappop(frontier)
        if s in closed:             continue      # entri usang
        if g > best_g[s] and mode != "greedy": continue
        if problem.goal_test(s):                  # uji saat diperluas
            return reconstruct(parent, s)
        closed.add(s)
        record_expansion(s)                       # untuk heat-map
        for a, s2, c in problem.successors(s):
            g2 = g + c
            if mode == "greedy":
                if s2 in best_g: continue
            elif g2 >= best_g.get(s2, INF):
                continue
            best_g[s2], parent[s2] = g2, (s, a)
            push(frontier, key(g2, s2), g2, s2)
    return failure

# kunci prioritas
key = g            (UCS)
key = (g + h, h)   (A*: seri diputus oleh h yang lebih kecil)
key = h            (Greedy)
```